Qualcomm Technologies, Inc.

Qualcomm® Sensing Hub (QSH) 4.0

API Reference

80-40939-19 Rev. AB

March 6, 2023

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:36:23 GMT
hyungchul.won@lge.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Confidential Distribution: Use or distribution of this item, in whole or in part, is prohibited except as expressly permitted by written agreement(s) and/or terms with Qualcomm Incorporated and/or its subsidiaries.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer (“export”) laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2023 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AA	January 2023	Initial release
AB	March 2023	Editorial updates

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:36:23 GMT
hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Contents

1 Introduction **4**
1.1 Purpose 4
1.2 Conventions 4
1.3 Technical assistance 4

2 QSH ASC Face Detection API **5**

3 QSH ASC Frame API **9**

4 QSH Audio Context API **16**

5 QSH Audio Data API **20**

6 QSH BLE API **24**

7 QSH Location API **45**

8 QSH Geofence API **53**

9 QSH Wi-Fi API **59**

10 QSH WWAN (modem) API **77**

11 Presence Detect API **88**

A References **90**
A.1 Related documents 90

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:36:23 GMT
hyungchul.won@lge.com

1 Introduction

1.1 Purpose

This document describes Qualcomm® Sensing Hub (QSH) APIs for Always Sensing Camera (ASC) Face Detection, ASC Frame, Audio Context, Audio Data, BLE, Location, Geofence, Wi-Fi, WWAN (modem), and Presence Detect data streams.

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `cp armcc armcpp`.

Code variables appear in angle brackets, for example, `<number>`.

Shading indicates content that has been added or changed in this revision of the document.

1.3 Technical assistance

For assistance or clarification on information in this document, open a technical support case at https://support.qualcomm.com/.

You will need to register for a Qualcomm ID account and your company must have support enabled to access our Case system.

Other systems and support resources are listed on https://qualcomm.com/support.

If you need further assistance, you can send an email to qualcomm.support@qti.qualcomm.com.

Qualcomm Confidential – May Contain Trade Secrets
2026-02-27 05:26:23 GMT
hyunchul.ryu@qti.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2 QSH ASC Face Detection API

QSH Always Sensing Camera (ASC) Face Detection API is available at adsp_proc/qsh/qsh_camera/pb /qsh_camera_fd.proto in ADSP build.

```
// @file qsh_camera_fd.proto
//
// Defines standard message types for the CAMERA face detection sensor
//
// Copyright (c) 2020-2021 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.
syntax = "proto2";
import "nanopb.proto";
import "sns_std_sensor.proto";

// Attribute requirements:
// SNS_STD_SENSOR_ATTRID_TYPE: "camera_face_detect"
// SNS_STD_SENSOR_ATTRID_AVAILABLE: true
// SNS_STD_SENSOR_ATTRID_API: "qsh_camera_fd.proto"
// SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_ON_CHANGE

// Custom attributes exposed by the sensor
enum qsh_camera_fd_attr_id
{
  option (nanopb_enumopt).long_names = false;
  // REQUIRED
  // uint32
  // The native streaming rate of the image sensor e.g. 30
  QSH_CAMERA_FD_ATTRID_NATIVE_FRAME_RATE                    = 1024;

  // REQUIRED
  // bool
  // Whether the underlying image sensor supports motion detection
  // This determines if the MOTION_BASED mode of delivery can be set
  // in the subscribe command
  QSH_CAMERA_FD_ATTRID_MOTION_BASED_DELIVERY_SUPPORTED      = 1025;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:30:12 GMT hyunchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
// Message IDs for camera face detect sensor
enum qsh_camera_fd_msgid {
  option (nanopb_enumopt).long_names = false;

  // Uses message: qsh_camera_fd_subscribe
  // Direction: From client to sensor
  // Purpose: Subscribes for face detection events
  // Expects QSH_CAMERA_FD_MSGID_QSH_ACK from the sensor
  QSH_CAMERA_FD_MSGID_QSH_CAMERA_FD_SUBSCRIBE               = 0x260; // 608

  // Uses message: qsh_camera_fd_subscribe_ack
  // Direction: From sensor to client
  // Purpose: ACK/NACK for the subscribe request
  // Client uses this as an indiction that the command was successful/completed
  // and it can send subsequent requests.
  QSH_CAMERA_FD_MSGID_QSH_CAMERA_FD_SUBSCRIBE_ACK           = 0x360; // 864

  // Does not use any message body
  // Direction: From client to sensor
  // Purpose: Unsubscribes for face detection events
  // Expects QSH_CAMERA_FD_MSGID_QSH_ACK from the sensor
  QSH_CAMERA_FD_MSGID_QSH_CAMERA_FD_UNSUBSCRIBE             = 0x261; // 609

  // Uses message: qsh_camera_fd_unsubscribe_ack
  // Direction: From sensor to client
  // Purpose: ACK/NACK for the unsubscribe request
  // Client uses this as an indiction that the command was successful/completed
  // and it can send subsequent requests.
  QSH_CAMERA_FD_MSGID_QSH_CAMERA_FD_UNSUBSCRIBE_ACK         = 0x361; // 865

  // Uses message: qsh_camera_fd_event
  // Direction: From sensor to client
  // Purpose: Reports face detection event
  QSH_CAMERA_FD_MSGID_QSH_CAMERA_FD_EVENT                   = 0x362; // 866
}

enum qsh_camera_fd_error_type {
  option (nanopb_enumopt).long_names = false;

  // No error - Successful operation
  QSH_CAMERA_FD_ERROR_TYPE_SUCCESS                          = 0;
  // General error without a specific reason
  QSH_CAMERA_FD_ERROR_TYPE_FAILURE_GENERAL                  = 1;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.kwon@zne.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
// Payload associated with QSH_CAMERA_FD_MSGID_QSH_CAMERA_FD_SUBSCRIBEACK
message qsh_camera_fd_subscribe_ack
{
  // The command was successful or not
  required qsh_camera_fd_error_type error                    = 1;
}

// Payload associated with QSH_CAMERA_FD_MSGID_QSH_CAMERA_FD_UNSUBSCRIBE_ACK
message qsh_camera_fd_unsubscribe_ack
{
  // The command was successful or not
  required qsh_camera_fd_error_type error                    = 1;
}

// Payload associated with QSH_CAMERA_FD_MSGID_QSH_CAMERA_FD_EVENT
enum qsh_camera_fd_event_type
{
  option (nanopb_enumopt).long_names = false;

  // This indicates that face detection was performed and a face is detected.
  QSH_CAMERA_FD_EVENT_TYPE_FACE_DETECTED                     = 1;

  // This indicates that face detection was performed and a face is not detected.
  QSH_CAMERA_FD_EVENT_TYPE_FACE_NOT_DETECTED                 = 2;

  // This indicates that face detection is temporarily suspended because the
  // shared image sensor is taken for the higher-priority non-island use
  // as main camera
  QSH_CAMERA_FD_EVENT_TYPE_SUSPENDED                         = 4;

  // This indicates that face detection is resumed because the shared
  // image sensor is now given back after non-island use as main camera
  QSH_CAMERA_FD_EVENT_TYPE_RESUMED                           = 8;
}

message qsh_camera_fd_event
{
  // The type of the FD event
  // it is derived from the bit values defined in enum qsh_camera_fd_event_type
  required uint32 event_type                                 = 1;
}

enum qsh_camera_fd_delivery_mode
{
  option (nanopb_enumopt).long_names = false;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 09:36:23 GMT hyungsul.kim@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // Face detection is triggered to run based on motion detected by sensor
  // if supported
  QSH_CAMERA_FD_DELIVERY_MODE_MOTION_BASED                  = 0;

  // Face detection is triggered to run periodically
  QSH_CAMERA_FD_DELIVERY_MODE_TIME_BASED                    = 1;
}

// Payload associated with QSH_CAMERA_FD_MSGID_QSH_CAMERA_FD_SUBSCRIBE
message qsh_camera_fd_subscribe
{
  // A bit-mask indicating which events are to be subscribed
  // For example, if both face and no-face events are to be subscribed
  // then the event_mask should be set as 0x3 (= 0x1 | 0x2)
  optional uint32                      event_mask              = 1 [default = 1];

  // TIME_BASED or MOTION_BASED
  optional qsh_camera_fd_delivery_mode delivery_mode           = 2 [default =
QSH_CAMERA_FD_DELIVERY_MODE_MOTION_BASED];

  // detections_per_delivery:
  //
  // Number of times face detections are performed per delivery
  // (applicable for both delivery_mode - MOTION_BASED & TIME_BASED)
  //
  // If delivery_mode is MOTION_BASED:
  //    This is the number of detections that will be performed each
  //    time a motion is detected
  //
  // If delivery_mode is TIME_BASED:
  //    At the specified delivery periodicity, face detection will be
  //    performed consecutively at the native streaming rate of the
  //    image sensor.
  optional uint32                      detections_per_delivery = 3 [default = 1];

  // delivery_period_in_ms:
  //
  // How often events will be delivered
  // (applicable when delivery_mode is set as TIME_BASED)
  // For example, if delivery mode is TIME_BASED, events_per_delivery
  // is 2 and delivery_period_in_ms is 1000, then every second, face
  // detection will be performed 2 times consecutively (each time 33ms
  // apart if the underlying sensor streams at 30fps natively). Out
  // of the 2 detections, either face/no face event will be raised
  // depending on the event_mask specified.
  optional uint32                      delivery_period_in_ms   = 4 [default =
1000];

}
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 09:36:23 GMT hyungchul.kim@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3 QSH ASC Frame API

QSH Always Sensing Camera (ASC) Frame API is available in the ADSP build at adsp_proc/qsh/qsh_camera/pb /qsh_camera_frame.proto.

```
// @file qsh_camera_frame.proto
//
// Defines standard message types for the CAMERA frame sensor
//
// Copyright (c) 2020-2021 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.
syntax = "proto2";
import "nanopb.proto";
import "sns_std_sensor.proto";

// Attribute requirements:
// SNS_STD_SENSOR_ATTRID_TYPE: "camera_frame"
// SNS_STD_SENSOR_ATTRID_AVAILABLE: true
// SNS_STD_SENSOR_ATTRID_API: "qsh_camera_frame.proto"
// SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_STREAMING
// QSH_CAMREA_FRAME_ATTRID_IMAGE_RESOLUTIONS
// QSH_CAMREA_FRAME_ATTRID_IMAGE_FORMATS
// QSH_CAMERA_FRAME_ATTRID_NATIVE_FRAME_RATE
// QSH_CAMERA_FRAME_ATTRID_MOTION_BASED_DELIVERY_SUPPORTED

// Possible image formats supported
enum qsh_camera_frame_image_format
{
  option (nanopb_enumopt).long_names = false;

  // YUV 420 format
  // Luma and chroma are stored in two different buffer planes
  // Chroma is subsampled two times in both horizontal and vertical
  // directions. Both luma and chroma are 8-bit per pixel in size
  QSH_CAMERA_FRAME_IMAGE_FORMAT_YUV420     = 0;

  // 8-bit luma only
  // Uses only a single buffer plane to store. Each pixel is 8-bit in size
  QSH_CAMERA_FRAME_IMAGE_FORMAT_8_BIT_LUMA = 1;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 09:30:12 GMT hyungchul.kim@qti.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
// Custom attributes exposed by the sensor
enum qsh_camera_frame_attr_id
{
  option (nanopb_enumopt).long_names = false;

  // REQUIRED
  // Integer
  // The native streaming rate of the image sensor e.g. 30 Hz
  QSH_CAMERA_FRAME_ATTRID_NATIVE_FRAME_RATE                        = 1024;

  // REQUIRED
  // bool
  // Whether the underlying image sensor supports motion detection
  // This determines if the MOTION_BASED mode of delivery can be set
  // in the subscribe command
  QSH_CAMERA_FRAME_ATTRID_MOTION_BASED_DELIVERY_SUPPORTED          = 1025;

  // REQUIRED
  // [{Integer,Integer}]
  // Supported image output resolutions (width, height)
  // e.g. [(640,480), (320,240), (160,120)]
  // The same underlying HW is used to support other type
  // of sensor data such as Face-Detection events. If the
  // underlying HW is already in use by Face-Detection client,
  // then the supported image resolutions may be limited to a
  // subset (e.g. 160x120)
  QSH_CAMERA_FRAME_ATTRID_IMAGE_RESOLUTIONS                        = 1026;

  // REQUIRED
  // [qsh_camera_frame_image_format]
  // The same underlying HW is used to support other type
  // of sensor data such as Face-Detection events. If the
  // underlying HW is already in use by Face-Detection client,
  // then the supported image format may be limited to a
  // subset (e.g. only 8_BIT_LUMA)
  QSH_CAMERA_FRAME_ATTRID_IMAGE_FORMATS                            = 1027;
}

// Message IDs for camera sensor
enum qsh_camera_frame_msgid
{
  option (nanopb_enumopt).long_names = false;

  ////////////////////////////////////////////////////////////////////////////
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // Uses message: qsh_camera_frame_subscribe
  // Direction: From client to sensor
  // Purpose: For client to subscribe for frame events
  // Expects QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_SUBSCRIBE_ACK from sensor
  QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_SUBSCRIBE                = 0x280; // 640

  // Uses message: qsh_camera_frame_subscribe_ack
  // Direction: From sensor to client
  // Purpose: ACK/NACK for the subscribe request
  // Client uses this as an indiction that the command was successful/completed
  // and it can send subsequent requests.
  QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_SUBSCRIBE_ACK            = 0x380; // 896

  ////////////////////////////////////////////////////////////////////////////

  // Does not use any message body
  // Direction: From client to sensor
  // Purpose: For client to request for frame events
  // Expects QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_UNSUBSCRIBE_ACK from sensor
  QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_UNSUBSCRIBE              = 0x281; // 641

  // Uses message: qsh_camera_frame_unsubscribe_ack
  // Direction: From sensor to client
  // Purpose: ACK/NACK for the unsubscribe request
  // Client uses this as an indiction that the command was successful/completed
  // and it can send subsequent requests.
  QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_UNSUBSCRIBE_ACK          = 0x381; // 897

  ////////////////////////////////////////////////////////////////////////////

  // Uses message: qsh_camera_frame_event
  // Direction: From sensor to client
  // Purpose: For sensor to report a frame event to client
  QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_EVENT                    = 0x482; //
1154

  ////////////////////////////////////////////////////////////////////////////

  // Uses message: qsh_camera_frame_release_buffer
  // Direction: From client to sensor
  // Purpose: For client to release frame buffer back to sensor that was
previously
  //          delivered to client
  //
  // For sensor to be able to deliver frames to client, client must release
buffers
  // back to sensor after they were consumed.
  QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_RELEASE_BUFFER           = 0x283; // 643
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 09:36:22 GMT hyungchul.yoon@zte.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // Uses message: qsh_camera_frame_release_buffer_ack
  // Direction: From sensor to client
  // Purpose: For sensor to acknowledge release buffer request
  QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_RELEASE_BUFFER_ACK       = 0x383; // 899
}

enum qsh_camera_frame_error_type
{
  option (nanopb_enumopt).long_names = false;

  // No error (successful operation)
  QSH_CAMERA_FRAME_ERROR_TYPE_SUCCESS                              = 0;
  // General error with no specific reason
  QSH_CAMERA_FRAME_ERROR_TYPE_FAILURE_GENERAL                      = 1;
}

// Payload associated with:
//   QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_SUBSCRIBE_ACK
message qsh_camera_frame_subscribe_ack
{
  // The command was successful or not
  required qsh_camera_frame_error_type error                       = 1;
}

// Payload associated with:
//   QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_UNSUBSCRIBE_ACK
message qsh_camera_frame_unsubscribe_ack
{
  // The command was successful or not
  required qsh_camera_frame_error_type error                       = 1;
}

// Payload associated with:
//   QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_RELEASE_BUFFER_ACK
message qsh_camera_frame_release_buffer_ack
{
  // The command was successful or not
  required qsh_camera_frame_error_type error                       = 1;
}

// Payload associated with:
//   QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_RELEASE_BUFFER
//   QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_EVENT (used in Frame Ready event)
message qsh_camera_frame_release_buffer
{
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // The virtual address of the buffer plane 0
  required uint32 addr_plane0                                        = 1;
  // The buffer ID of plane 0 - used by sensor when buffer is returned
  required uint32 buffer_id_plane0                                   = 2;
  // The virtual address of the buffer plane 1 (if applicable, e.g. YUV420)
  optional uint32 addr_plane1                                        = 3;
  // The buffer ID of plane 1 - used by sensor when buffer is returned
  optional uint32 buffer_id_plane1                                   = 4;
}

// delivery_mode:
//
// There are two modes where frames can be delivered:
//  1) Motion-based:
//     Raw sensor frame(s) will be delivered only when a motion is detected
//     This is only supported when the underlying image sensor supports
//     in-sensor optical motion detection. The sensor exposes the
//     QSH_CAMERA_FRAME_ATTRID_MOTION_BASED_DELIVERY_SUPPORTED to indicate
//     whether this mode is supported
//  2) Time-based:
//     Raw sensor frames will be delivered according to periodically. Sensor
//     will wake up according to the specified periodicity ('delivery_per_sec')
//     and deliver n number of frames where n is specified by the
//     'frames_per_delivery' fields in the subscribe arguments.
//
enum qsh_camera_frame_delivery_mode
{
  option (nanopb_enumopt).long_names = false;

  QSH_CAMERA_FRAME_DELIVERY_MODE_MOTION_BASED                        = 0;
  QSH_CAMERA_FRAME_DELIVERY_MODE_TIME_BASED                          = 1;
}

enum qsh_camera_frame_event_type
{
  option (nanopb_enumopt).long_names = false;

  // This indicates that a frame is ready
  QSH_CAMERA_FRAME_EVENT_TYPE_FRAME_READY                            = 1;
  // This indicates that frame generation is temporarily suspended because the
shared
  // image sensor is taken for the higher-priority non-island use as main camera
  QSH_CAMERA_FRAME_EVENT_TYPE_SUSPENDED                              = 2;
  // This indicates that frame generation is resumed because the shared
  // image sensor is now given back after non-island use as main camera
  QSH_CAMERA_FRAME_EVENT_TYPE_RESUMED                                = 4;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-23 05:36:23 GMT hyungchul.kim@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
// Request payload for 'Subscribe'
// Payload associated with QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_SUBSCRIBE
message qsh_camera_frame_subscribe
{
  // A bit-mask indicating which events are to be subscribed
  required uint32                          event_mask             = 1 [default =
1];

  required qsh_camera_frame_delivery_mode   delivery_mode          = 2;

  // frames_per_delivery:
  //
  // Number of frames per delivery
  // (applicable for both delivery_mode - MOTION_BASED & TIME_BASED)
  //
  // If delivery_mode is MOTION_BASED:
  //    This is the number of frames that will be delivered each
  //    time a motion is detected
  //    Minimum value allowed: 1
  //    Maximum value allowed: No limit
  //
  // If delivery_mode is TIME_BASED:
  //    At the specified delivery periodicity, frames will be delivered
consecutively at
  //    the native streaming rate of the image sensor.
  //    Minimum value allowed: 1
  //    Maximum value allowed: No limit
  required uint32                          frames_per_delivery    = 3;

  // delivery_period_in_ms:
  //
  // How often frames will be delivered
  // (applicable when delivery_mode is set as TIME_BASED)
  // For example, if delivery mode is TIME_BASED, frames_per_delivery is 2
  // and delivery_period_in_ms is 1000, every second, 2 frames will be delivered
  // (each frame 33ms apart if the underlying sensor streams at 30fps natively)
  optional uint32                          delivery_period_in_ms  = 4;

  // image_format:
  // The desired image format - needs to be within the supported image formats
  required qsh_camera_frame_image_format    image_format           = 5;

  // image_width:
  // The desired image width - needs to be within the supported image resolution
  required uint32                          image_width            = 6;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // image_height:
  // The desired image height - needs to be within the supported image resolution
  required uint32                          image_height           = 7;

  // buffer_stride_plane0:
  // The buffer stride for plane 0 in number of bytes
  required uint32                          buffer_stride_plane0   = 8;

  // buffer_stride_plane1:
  // The buffer stride for plane 1 in number of bytes
  // This is optional if the specified image format contains 1 single plane
  // e.g. 8_BIT_LUMA
  optional uint32                          buffer_stride_plane1   = 9;
}

// Event payload for 'Frame Event'
// Payload associated with QSH_CAMERA_FRAME_MSGID_QSH_CAMERA_FRAME_EVENT
message qsh_camera_frame_event
{
  // The type of the frame event
  // it is derived from the bit values defined in enum qsh_camera_frame_event_type
  required uint32                          event_type             = 1;

  // The frame buffer information in the case of the 'FRAME_READY' event
  optional qsh_camera_frame_release_buffer frame_buffer           = 2;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4 QSH Audio Context API

QSH Audio Context API is available in the ADSP build at adsp_proc/qsh/qsh_audio/audio_sns/audio_context/pb/qsh_audio_context.proto.

```
// @file qsh_audio_context.proto
//
// Sensor provides access to audio context events.
//
// This Sensor will create a connection to the remote process where Audio context
services will run.
// A new instance will be created for each client.
// Audio context services will be started in remote process upon receipt of the
first client request.
// Subsequent requests will be sent to the same service.
// The connection will be closed after all the client requests are removed.
//
// If an error event is received from the remote process then all the clients will
be informed.
// It is then the responsibility of the client to close the data stream, and re-
open a new one if desired.
//
// Copyright (c) 2020-2021 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.

syntax = "proto2";
import "nanopb.proto";
import "sns_std.proto";

// Attribute requirements:
// SNS_STD_SENSOR_ATTRID_TYPE: "audio_context"
// SNS_STD_SENSOR_ATTRID_AVAILABLE: true if connected to ADSP
// SNS_STD_SENSOR_ATTRID_API: "qsh_audio_context.proto"
// SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_ON_CHANGE
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.woo@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
// In following situation an error event,
// SNS_STD_MSGID_SNS_STD_ERROR_EVENT, with payload sns_std_error_event, having
// appropriate error code, will be raised:
// 1. if remote process is not able the handle the request.
// 2. if there is any IPC error.

enum sns_audio_context_msgid
{
  option (nanopb_enumopt).long_names = false;

  // Request message to enable context events
  // if client sends multiple request then the latest request will be handled and
previous requests will be removed.
  QSH_AUDIO_CONTEXT_MSGID_QSH_AUDIO_CONTEXT_REQ = 512;

  // Request message to query supported context events
  QSH_AUDIO_CONTEXT_MSGID_QSH_AUDIO_CONTEXT_QUERY = 513;

  // Asynchronously received context event message
  QSH_AUDIO_CONTEXT_MSGID_QSH_AUDIO_CONTEXT_EVENTS = 1024;

  // Asynchronously received message to inform supported contexts
  QSH_AUDIO_CONTEXT_MSGID_QSH_AUDIO_CONTEXT_LIST = 1025;
}

//context configuration payload.
//refer to acd_context_specific_generic_reg_info_t in
qsh\qsh_audio\audio_api\acd_api.h
message context_config
{
  //GUID of the context to enable or disable
  //refer to qsh\qsh_audio\audio_api\contexts_api.h
  required uint32 context_id = 1;

  //threshold for context detection. range 0 to 100.
  //event is raised if threshold is met.
  //when detected threshold crosses the configured threshold, client is informed
with event type: QSH_AUDIO_CONTEXT_EVENT_DETECTED
  //now client is continously informed about the detected threshold even if it
gradually decreases and goes below the configured threshold.
  //once detected threshold goes below the minimum configured threshold across all
the clients then event is considered STOPPED and
  //clients are notified with event_type: QSH_AUDIO_CONTEXT_EVENT_STOPPED
  required uint32 threshold = 2;

  //step size in threshold
  //step_size can be used by clients to configure the frequency of
QSH_AUDIO_CONTEXT_EVENT_DETECTED event.
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungkyu.kim@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  //client is guaranteed to receive the DETECTED event if there is variation in
detected-threshold by step_size amount.
  //client may also get an event if threshold vaires within step_size.
  required uint32 step_size = 3;
}

enum context_event_type {
  option (nanopb_enumopt).long_names = false;

  // invalid event type
  QSH_AUDIO_CONTEXT_EVENT_INVALID = 0;

  // context detection is stopped,
  // detected threshold went below the minimum configured threshold from all the
clients.
  QSH_AUDIO_CONTEXT_EVENT_STOPPED = 1;

  // context is detected,
  // detected threshold crossed the configured threshold or
  // detected threshold changed significantly after crossing the configured
threshold.
  QSH_AUDIO_CONTEXT_EVENT_DETECTED = 2;
}

//context event payload.
//refer to acd_context_specific_generic_event_t in
qsh\qsh_audio\audio_api\acd_api.h
message context_event
{
  //GUID of the detected context
  //refer to qsh\qsh_audio\audio_api\contexts_api.h
  required uint32 context_id = 1;

  //Context Detected is Stopped or Detected
  required context_event_type event_type = 2;

  //detected threshold of the context. range 0 to 100
  required uint32 threshold = 3;

  //timestamp at which context event started.
  //this is deferent than the sns_sensor_event:timestamp
  /* e.g.
  * if Baby starts crying at t = 0 sec but detected threshold is crossed at t = 2
sec
  * then start_timestamp will be t = 0 sec and sns_sensor_event:timestamp will be
t = 2 sec.
  * sns_sensor_event:timestamp is the time at which threshold is evaluated.
  * and start_timestamp is the time at which event actually started occuring.
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  * In case of ongoing events (e.g baby is continously crying) start_timestamp
remains same
  * (the first time when baby started crying).
  */
  required uint64 start_timestamp = 4;
}

//request payload (QSH_AUDIO_CONTEXT_MSGID_REQ)
message qsh_audio_context_req
{
  //configuration for each context
  /* maximum three contexts can be configured per client. */
  repeated context_config config = 1 [(nanopb).max_count = 3];
}

//event payload (QSH_AUDIO_CONTEXT_MSGID_EVENTS)
message qsh_audio_context_events
{
  //event for each detected context
  repeated context_event events = 1;
}

//payload (QSH_AUDIO_CONTEXT_MSGID_CONTEXT_LIST)
message qsh_audio_context_list
{
  //array of context ids
  repeated uint32 context_id = 1;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

5 QSH Audio Data API

QSH Audio Data API is available in the ADSP build at adsp_proc/qsh/qsh_audio/audio_sns/audio_data/pb/qsh_audio_data.proto.

```
// @file qsh_audio_data.proto
//
// Sensor provides access to audio pcm data.
//
// Copyright (c) 2021 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.

syntax = "proto2";
import "nanopb.proto";
import "sns_std.proto";

// Attribute requirements:
// SNS_STD_SENSOR_ATTRID_TYPE: "audio_data"
// SNS_STD_SENSOR_ATTRID_AVAILABLE: true if connected to ADSP
// SNS_STD_SENSOR_ATTRID_API: "qsh_audio_data.proto"
// SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_STREAMING

// In following situation an error event,
// SNS_STD_MSGID_SNS_STD_ERROR_EVENT, with payload sns_std_error_event, having
// appropriate error code, will be raised:
// 1. if remote process is not able the handle the request.
// 2. if there is any IPC error.

enum sns_audio_data_msgid
{
  option (nanopb_enumopt).long_names = false;

  // Message to set up audio data config
  // New instance is created for every new client
  QSH_AUDIO_DATA_MSGID_QSH_AUDIO_DATA_CONFIG = 512;

  // Message to enable or disable audio data flow
  QSH_AUDIO_DATA_MSGID_QSH_AUDIO_DATA_FLOW = 513;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yoon@qce.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // Event for media format of data
  QSH_AUDIO_DATA_MSGID_QSH_AUDIO_DATA_MEDIA_FORMAT_EVENT = 768;

  // Event on reception of data
  QSH_AUDIO_DATA_MSGID_QSH_AUDIO_DATA_EVENT = 1024;

}

// Stream types for pcm data

enum qsh_audio_data_stream_type_t
{
   option (nanopb_enumopt).long_names = false;

   // Invalid
   QSH_AUDIO_DATA_STREAM_TYPE_INVALID = 0;

   // Noise suppression is not required
   QSH_AUDIO_DATA_STREAM_TYPE_PCM_RAW = 1;

   // Noise suppression is required
   QSH_AUDIO_DATA_STREAM_TYPE_PCM_NS = 2;
}

//payload (QSH_AUDIO_DATA_MSGID_QSH_AUDIO_DATA_CONFIG)
//when sensor cients sends this configuration then an audio-capture session is
setup in the ADSP, which keeps buffering the audio data in a buffer till the data-
flow is enabled.
message qsh_audio_data_config
{
   // history data in millisecond
   // this can be used to configure the required amount of past data at data-flow
enable.
   // this may not be honored always, there can be some cases when ADSP didn't get
chance to buffer the required history data and data-flow is enabled. ADSP may also
clear its buffer based on some other internal trigger.
   required uint32 history_ms = 1 [default = 0];

   // stream type for pcm data
   // this can be used to select either noise supressed or raw mic data.
   required qsh_audio_data_stream_type_t stream_type = 2  [default =
QSH_AUDIO_DATA_STREAM_TYPE_PCM_RAW];

}

//payload (QSH_AUDIO_DATA_MSGID_QSH_AUDIO_DATA_FLOW)
```

Qualcomm Confidential - May Contain Trade Secrets 2026-01-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
//when data flow is enabled then ADSP starts sending the buffered data according
to the configured history, this data is sent faster than real-time. Once the
history data is sent out then real time data starts flowing.
message qsh_audio_data_flow
{
   //start/stop data flow - 0 : disable and 1 : enable
   required bool data_flow_enable  = 1 [default = false];
}

enum qsh_audio_data_endian_t
{
   option (nanopb_enumopt).long_names = false;

   // Invalid
   QSH_AUDIO_DATA_INVALID_VALUE = 0;

   // Little Endian
   QSH_AUDIO_DATA_PCM_LITTLE_ENDIAN = 1;

   // Big Endian
   QSH_AUDIO_DATA_PCM_BIG_ENDIAN = 2;
}

//payload (QSH_AUDIO_DATA_MSGID_QSH_AUDIO_DATA_MEDIA_FORMAT_EVENT)
//this is an event to inform about the media format of the pcm data sent in
QSH_AUDIO_DATA_MSGID_QSH_AUDIO_DATA_EVENT event.
//this is usually sent once before the first data-packet which is sent after the
data-flow enable.
message qsh_audio_data_media_format_event
{
   //sampling rate in samples per second
   //currenty support only 16kHz
   required uint32 sampling_rate = 1 [default = 16000];

   //Number of channels
   required uint32 num_channels = 2 [default = 2];

   //Number of bits used to store each sample per channel
   required uint32 bits_per_sample = 3 [default = 16];

   //PCM samples are stored in little or big endian format
   required qsh_audio_data_endian_t endianness = 4 [default =
QSH_AUDIO_DATA_PCM_LITTLE_ENDIAN];

   required qsh_audio_data_stream_type_t stream_type = 5 [default =
QSH_AUDIO_DATA_STREAM_TYPE_PCM_RAW];
}
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:36:25 GMT hyunghul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
//payload (QSH_AUDIO_DATA_MSGID_QSH_AUDIO_DATA_EVENT)
//event to send the pcm data
message qsh_audio_data_event
{
  //this payload carries the timestamp of the first PCM sample
  required bytes buffer = 1;
}
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:36:23 GMT
hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

6 QSH BLE API

QSH BLE API is available at adsp_proc/qsh/qsh_ble/pb/qsh_ble_ext.proto in the ADSP build.

```
// @file qsh_ble_ext.proto
//
// Defines the API for qsh ble extended sensor.
//
// Copyright (c) 2022 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.

syntax = "proto2";
import "nanopb.proto";
import "sns_std.proto";
import "sns_std_sensor.proto";
import "sns_std_type.proto";

/**
 * ble_ext sensor Attribute Requirements:
 * SNS_STD_SENSOR_ATTRID_TYPE: "ble_ext"
 * SNS_STD_SENSOR_ATTRID_API: "qsh_ble_ext.proto"
 * SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_ON_CHANGE
 * SNS_STD_SENSOR_ATTRID_TRANSPORT_MTU_SIZE
*/

/**
 * ble_ext sensor provides functions of BLE advertising and BLE scanning.
 *
 * BLE advertising:
* 1) When sensor client wants to use advertising function, sensor client should
firstly
 *  look up the attribute SNS_STD_SENSOR_ATTRID_TRANSPORT_MTU_SIZE to get the
 *  maximum advertising data length.
*  Currently the value of atribute SNS_STD_SENSOR_ATTRID_TRANSPORT_MTU_SIZE is
229, but
 *  the value might be changed in the future.
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 09:36:23 GMT
hyungchun@qualcomm.com

```
 * 2) Sensor client can send
QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_ADV_START_ADVERTISING
 *  to start advertising and can send
QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_ADV_STOP_ADVERTISING
 *  to stop advertising.
 *  For QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_ADV_START_ADVERTISING,
 *  if no advertising is started, ble_ext sensor will start advertising;
 *  if the advertising is ongoing and the new parameter is different, ble_ext
sensor will
 *  reconfigure advertising with the new parameter, advertising data and duration;
 *  if the advertising is ongoing and the new parameter is same, ble_ext sensor
will directly
 *  update advertising data and duration.
 * 3) Each data stream will have a corresponding advertising set in ble_ext
sensor. Sensor
 *  client can create multiple data streams for multiple advertising. If there is
no
 *  available advertising set, QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_ADV_STARTED_EVENT
with status
 *  QSH_BLE_EXT_ADV_STATUS_FAILED_TOO_MANY_ADVERTISERS will be responded to sensor
client.
 *
 * BLE scanning:
 * 1) ble_ext sensor supports setting up scan filters and starting scan.
 * 2) Sensor client can send QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_SCAN_START_SCAN
 *  to start scan and can send QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_SCAN_STOP_SCAN
 *  to stop scan.
 * 3) ble_ext will send the filtered advertising data to sensor client by using
 *  QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_SCAN_RESULT_EVENT.
 *
 *
 * Sensor client should not send next request until the ACK for current request is
received.
 *
 * When BT SOC is turned on or off, attribute SNS_STD_SENSOR_ATTRID_AVAILABLE will
be published
 * to notify the availability of ble_ext sensor. Sensor client should keep the
suid stream
 * open to receive the availability of ble_ext sensor.
 * After unavailable is published by ble_ext sensor:
 *  1) Sensor client will receive an unavailable notification through the suid
stream.
 *  2) Sensor client will also receive a SNS_STD_MSGID_SNS_STD_ERROR_EVENT with
error code
 *   SNS_STD_ERROR_INVALID_STATE if the data stream to ble_ext is already created.
Sensor client
 *   should remove the data stream after receiving this error event.
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:26:33 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
 *  3) In ble_ext, advertising and scanning status will be reset. No advertising
and scanning
 *   will be started on its own till sensor client restarts it again once the
available is published.
 *
 * An error event SNS_STD_MSGID_SNS_STD_ERROR_EVENT, with payload message
 * sns_std_error_event, having appropriate error code, will be sent to sensor
 * client when error happened in ble_ext sensor code. If error code is
 * SNS_STD_ERROR_INVALID_STATE, sensor client shall remove the data stream to
 * deactivate ble_ext sensor.
 */

enum qsh_ble_ext_msgid
{
  option (nanopb_enumopt).long_names = false;

 // BLE advertising:

  /**
   * Uses payload message: qsh_ble_ext_req_adv_start_advertising
   * Direction: From client to ble_ext sensor.
   * Purpose: To start advertising.
   * Expectation: QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_ADV_STARTED_EVENT.
   */
  QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_ADV_START_ADVERTISING  = 621;

  /**
   * Uses payload message: none
   * Direction: From client to ble_ext sensor.
   * Purpose: To stop advertising.
   * Expectation: QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_ADV_STOPPED_EVENT.
   */
  QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_ADV_STOP_ADVERTISING  = 622;

  /**
   * Uses payload message: qsh_ble_ext_req_adv_set_random_address
   * Direction: From client to ble_ext sensor
   * Purpose: To set advertising set random address to BT SOC.
   * By default, anonymous advertising is used. If random address is provided by
using
   * this message request before the first START_ADVERTISING request after data
stream
   * is created, all following advertising will use random address.
   * The provided random address is only valid in current data stream. Once data
stream
   * is destroyed, the random address will be cleared.
   * Expectation: QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_ADV_RANDOM_ADDRESS_SET_EVENT.
   */
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.kim@hne.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_ADV_SET_RANDOM_ADDRESS = 623;

/**
 * Uses payload message: qsh_ble_ext_evt_adv_started_event
 * Direction: From ble_ext sensor to sensor client.
 * Purpose: To ACK for QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_ADV_START_ADVERTISING.
 */
QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_ADV_STARTED_EVENT = 1121;

/**
 * Uses payload message: qsh_ble_ext_evt_adv_stopped_event
 * Direction: From ble_ext sensor to sensor client.
 * Purpose: To ACK for QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_ADV_STOP_ADVERTISING or
 * unsolicited notification to client when advertising duration expires.
 */
QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_ADV_STOPPED_EVENT = 1122;

/**
 * Uses payload message: qsh_ble_ext_evt_adv_random_address_set_event
 * Direction: From ble_ext sensor to sensor client.
 * Purpose: To ACK for QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_ADV_SET_RANDOM_ADDRESS.
 */
QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_ADV_RANDOM_ADDRESS_SET_EVENT = 1123;


// BLE scanning:

/**
 * Uses payload message: qsh_ble_ext_req_scan_start_scan
 * Direction: From client to ble_ext sensor
 * Purpose: To start scan for the specified advertising data.
 * Expectation: QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_SCAN_STARTED_EVENT.
 */
QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_SCAN_START_SCAN = 641;

/**
 * Uses payload message: none
 * Direction: From client to ble_ext sensor
 * Purpose: To stop scan for the specified advertising data.
 * Expectation: QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_SCAN_STOPPED_EVENT.
 */
QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_SCAN_STOP_SCAN = 642;

/**
 * Uses payload message: qsh_ble_ext_evt_scan_started_event
 * Direction: From ble_ext sensor to sensor client
 * Purpose: To ACK for QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_SCAN_START_SCAN.
 */
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.wonlange.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_SCAN_STARTED_EVENT = 1141;

  /**
   * Uses payload message: qsh_ble_ext_evt_scan_stopped_event
   * Direction: From ble_ext sensor to sensor client
   * Purpose: To ACK for QSH_BLE_EXT_MSGID_QSH_BLE_EXT_REQ_SCAN_STOP_SCAN.
   */
  QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_SCAN_STOPPED_EVENT = 1142;

  /**
   * Uses payload message: qsh_ble_ext_evt_scan_result_event
   * Direction: From ble_ext sensor to sensor client
   * Purpose: To deliver scan result to sensor client.
   */
  QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_SCAN_RESULT_EVENT = 1143;

  /**
   * Uses payload message: qsh_ble_ext_evt_raw_hci_event
   * Direction: From ble_ext sensor to sensor client
   * Purpose: To deliver raw hci event to sensor client.
   */
  QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_RAW_HCI_EVENT = 1151;
}

enum qsh_ble_ext_adv_param_interval_constants
{
  option (nanopb_enumopt).long_names = false;
  option allow_alias = true;

  /**
   * Advertise on low frequency, around every 1000ms. This is the default and
   * preferred advertising mode as it consumes the least power.
   */
  QSH_BLE_EXT_ADV_PARAM_INTERVAL_HIGH = 1000;

  /**
   * Advertise on medium frequency, around every 250ms. This is balanced
   * between advertising frequency and power consumption.
   */
  QSH_BLE_EXT_ADV_PARAM_INTERVAL_MEDIUM = 250;

  /**
   * Perform high frequency, low latency advertising, around every 100ms. This
   * has the highest power consumption and should not be used for continuous
   * background advertising.
   */
  QSH_BLE_EXT_ADV_PARAM_INTERVAL_LOW = 100;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yun@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  /**
   * Minimum value for advertising interval in ms.
   */
  QSH_BLE_EXT_ADV_PARAM_INTERVAL_MIN = 100;

  /**
   * Maximum value for advertising interval in ms.
   */
  QSH_BLE_EXT_ADV_PARAM_INTERVAL_MAX = 10485759;
}

enum qsh_ble_ext_adv_param_tx_power_constants
{
  option (nanopb_enumopt).long_names = false;

  /**
   * Advertise using the lowest transmission (TX) power level. Low transmission
   * power can be used to restrict the visibility range of advertising packets.
   */
  QSH_BLE_EXT_ADV_PARAM_TX_POWER_ULTRA_LOW = -21;

  /**
   * Advertise using low TX power level.
   */
  QSH_BLE_EXT_ADV_PARAM_TX_POWER_LOW = -15;

  /**
   * Advertise using medium TX power level.
   */
  QSH_BLE_EXT_ADV_PARAM_TX_POWER_MEDIUM = -7;

  /**
   * Advertise using high TX power level. This corresponds to largest visibility
   * range of the advertising packet.
   */
  QSH_BLE_EXT_ADV_PARAM_TX_POWER_HIGH = 1;

  /**
   * Minimum value for TX power.
   */
  QSH_BLE_EXT_ADV_PARAM_TX_POWER_MIN = -127;

  /**
   * Maximum value for TX power.
   */
  QSH_BLE_EXT_ADV_PARAM_TX_POWER_MAX = 20;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungschul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
enum qsh_ble_ext_adv_data_type_constants
{
  option (nanopb_enumopt).long_names = false;

  QSH_BLE_EXT_ADV_TYPE_SERVICE_UUID_16_BIT_PARTIAL = 2;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_UUID_16_BIT_COMPLETE = 3;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_UUID_32_BIT_PARTIAL = 4;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_UUID_32_BIT_COMPLETE = 5;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_UUID_128_BIT_PARTIAL = 6;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_UUID_128_BIT_COMPLETE = 7;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_DATA_UUID_16_BIT = 22;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_DATA_UUID_32_BIT = 32;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_DATA_UUID_128_BIT = 33;
  QSH_BLE_EXT_ADV_TYPE_MANUFACTURER_SPECIFIC_DATA = 255;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_SOLICITATION_UUID_16_BIT = 20;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_SOLICITATION_UUID_32_BIT = 31;
  QSH_BLE_EXT_ADV_TYPE_SERVICE_SOLICITATION_UUID_128_BIT = 21;
}

message qsh_ble_ext_advertising_parameters
{
  /**
   * Bluetooth LE Advertising interval, in 1ms unit. Valid range is from
   * 100 to 10,485,759. Recommended values are:
   * {QSH_BLE_EXT_ADV_PARAM_INTERVAL_HIGH},
{QSH_BLE_EXT_ADV_PARAM_INTERVAL_MEDIUM},
   * or {QSH_BLE_EXT_ADV_PARAM_INTERVAL_LOW}.
   * Default value is {QSH_BLE_EXT_ADV_PARAM_INTERVAL_MEDIUM} if this is not set.
   */
  optional uint32 interval = 1 [default = 250];

  /**
   * The transmission power of Bluetooth LE Advertising, in dBm. The valid
   * range is [-127, 1] Recommended values are:
   * {QSH_BLE_EXT_ADV_PARAM_TX_POWER_ULTRA_LOW},
   * {QSH_BLE_EXT_ADV_PARAM_TX_POWER_LOW},
   * {QSH_BLE_EXT_ADV_PARAM_TX_POWER_MEDIUM},
   * or {QSH_BLE_EXT_ADV_PARAM_TX_POWER_HIGH}.
   * Default value is {QSH_BLE_EXT_ADV_PARAM_TX_POWER_MEDIUM} if this is not set.
   */
  optional sint32 tx_power_level = 2 [default = -7];

  /**
   * Whether TX power should be included in the extended header.
   * Default value is false if this is not set.
   */
  optional bool include_tx_power = 3 [default = false];
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:13 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  /**
   * When set to true, advertising set will advertise legacy advertisements.
   * Note: leagacy advertising doesn't support anonymous advertising, so
   * public own address will be used by default if random address is not provided.
   */
  optional bool legacy = 4 [default = false];
}

/**
 * uuid is always in 128-bit format and little endian.
 * For bluetooth uuids:
 *   The BASE_UUID is "00000000-0000-1000-8000-00805F9B34FB".
 *   If the uuid is 16-bit 0xTTTT, then
 *   uuid_low will be 0x8000-00805F9B34FB and uuid_high will be 0x0000TTTT-0000-
1000.
 *   If the uuid is 32-bit 0xTTTTTTTT, then
 *   uuid_low will be 0x8000-00805F9B34FB and uuid_high will be 0xTTTTTTTT-0000-
1000.
 */
message qsh_ble_ext_uuid
{
  required fixed64 uuid_low = 1;
  required fixed64 uuid_high = 2;
}

message qsh_ble_ext_service_data
{
  required qsh_ble_ext_uuid uuid = 1;

  required bytes data = 2;
}

message qsh_ble_ext_manufacturer_data
{
  required uint32 manufacturer_id = 1;

  required bytes data = 2;
}

/**
 * Advertise data packet container for BLE advertising.
 * Each advertise data structure has a Length field of one byte,
 * an Advertise Type field of one byte and an Advertise Data field of Length - 1
 * bytes. Since the Length field only has one byte, the total length of
 * each advertise data structure shall not exceed 256 (1 byte for length field +
255).
 * OVERHEAD_BYTES_PER_FIELD = 2 ( one byte for Length field and
 *                                and one byte for Advertise Type field)
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT byungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
 * UUID length is 2, 4 or 16
 *
 * The total size of qsh_ble_ext_advertise_data is calculated in such below way:
 * 1) service_data_s length is
 *    for (service_data : service_data_s) {
 *      service_data_len += OVERHEAD_BYTES_PER_FIELD +
UUID_length(service_data.uuid) +
 *          length(service_data.data); // service_data_len shall be <=256
 *      total_size += service_data_len;
 *    }
 * 2) manufacturer_data_s length is
 *    for (manufacturer_data : manufacturer_data_s) {
 *      manufacturer_data_len += OVERHEAD_BYTES_PER_FIELD + 2 +
 *          length(manufacturer_data.data); // manufacturer_data_len shall be
<=256
 *      total_size += manufacturer_data_len;
 *    }
 * 3) service_uuid_s length is
 *      get num_of_uuid_2_bytes, num_of_uuid_4_bytes and num_of_uuid_16_bytes from
 *      service_uuid_s;
 *      if (num_of_uuid_2_bytes > 0) {
 *        uuid_len += OVERHEAD_BYTES_PER_FIELD +
 *            2 * num_of_uuid_2_bytes; // uuid_len shall be <=256
 *        total_size += uuid_len;
 *      }
 *      if (num_of_uuid_4_bytes > 0) {
 *        uuid_len += OVERHEAD_BYTES_PER_FIELD +
 *            4 * num_of_uuid_4_bytes // uuid_len shall be <=256
 *        total_size += uuid_len;
 *      }
 *      if (num_of_uuid_16_bytes > 0) {
 *        uuid_len += OVERHEAD_BYTES_PER_FIELD +
 *            16 * num_of_uuid_16_bytes // uuid_len shall be <=256
 *        total_size += uuid_len;
 *      }
 * 4) service_solicitation_uuid_s length is
 *      get num_of_uuid_2_bytes, num_of_uuid_4_bytes and num_of_uuid_16_bytes from
 *      service_solicitation_uuid_s;
 *      if (num_of_uuid_2_bytes > 0) {
 *        uuid_len += OVERHEAD_BYTES_PER_FIELD +
 *            2 * num_of_uuid_2_bytes; // uuid_len shall be <=256
 *        total_size += uuid_len;
 *      }
 *      if (num_of_uuid_4_bytes > 0) {
 *        uuid_len += OVERHEAD_BYTES_PER_FIELD +
 *            4 * num_of_uuid_4_bytes // uuid_len shall be <=256
 *        total_size += uuid_len;
 *      }
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:30:23 GMT hyung.shin.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
 *      if (num_of_uuid_16_bytes > 0) {
 *        uuid_len += OVERHEAD_BYTES_PER_FIELD +
 *            16 * num_of_uuid_16_bytes // uuid_len shall be <=256
 *        total_size += uuid_len;
 *      }
 *
 * total_size is the calculated size of qsh_ble_ext_advertise_data.
 *
 */
message qsh_ble_ext_advertise_data
{
  /**
   * service data list to be advertised.
   */
  repeated qsh_ble_ext_service_data service_data_s = 1;

  /**
   * manufacturer specific data list.
   */
  repeated qsh_ble_ext_manufacturer_data manufacturer_data_s = 2;

  /**
   * service uuid list to be advertised.
   */
  repeated qsh_ble_ext_uuid service_uuid_s = 3;

  /**
   * service solicitation uuid list to be advertised.
   */
  repeated qsh_ble_ext_uuid service_solicitation_uuid_s = 4;
}

/**
 * The calculated total size will be:
 * total_size = total size of advertise_data or length of raw_advertise_data;
 * if (include_local_name)
 *   total_size += 28
 *
 * total_len is the calculated size of qsh_ble_ext_advertise_data_union.
 */
message qsh_ble_ext_advertise_data_union
{
  option (nanopb_msgopt).no_unions=true;

  oneof data {
    qsh_ble_ext_advertise_data advertise_data = 1;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    /**
     * raw advertising data bytes which are built according to
     * "BT core spec, Vol 3, Part C, 11 ADVERTISING AND SCAN RESPONSE DATA
FORMAT".
     */
    bytes raw_advertise_data = 2;
  }

  /**
   * If include_local_name is true, 28 bytes will be consumed in advertising data.
   * Default value is false if this is not set.
   */
  optional bool include_local_name = 3;
}

message qsh_ble_ext_req_adv_start_advertising
{
  /**
   * advertising parameters.
   */
  optional qsh_ble_ext_advertising_parameters parameters = 1;

  /**
   * Advertisement data to be broadcasted.
   * The calculated total size must not exceed the value of
max_advertising_data_len.
   * max_advertising_data_len is published via
   * SNS_STD_SENSOR_ATTRID_TRANSPORT_MTU_SIZE atrribute.
   */
  required qsh_ble_ext_advertise_data_union advertise_data_union = 2;

  /**
   * Advertising duration, in 1ms unit. Valid range is from 10 to 655,350.
   * Recommended value is a multiple of 10.
   * After duration timeout, advertising will be stopped in ble_ext
   * automatically. 0 means advertising should continue until stopped explicitly
   * by sensor client.
   */
  required uint32 duration = 3;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:30:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_ble_ext_req_adv_set_random_address
{
  /**
   * random address format is defined in core spec [Vol 6] Part B, Section 1.3.2.
   * The LSB of random address should be put at address[0].
   */
  required bytes address = 1;
}

enum qsh_ble_ext_adv_status
{
  option (nanopb_enumopt).long_names = false;

  /**
   * The requested operation was successful.
   */
  QSH_BLE_EXT_ADV_STATUS_SUCCESS = 0;

  /**
   * Failed to start advertising as the advertise data to be broadcasted is too
   * large.
   */
  QSH_BLE_EXT_ADV_STATUS_FAILED_DATA_TOO_LARGE = 1;

  /**
   * Failed to register advertiser because no advertising instance is available.
   */
  QSH_BLE_EXT_ADV_STATUS_FAILED_TOO_MANY_ADVERTISERS = 2;

  /**
   * Operation failed due to an internal error.
   */
  QSH_BLE_EXT_ADV_STATUS_FAILED_INTERNAL_ERROR = 3;

  /**
   * Fails to start advertising as the parameters set by
   * sensor client are incorrect.
   */
  QSH_BLE_EXT_ADV_STATUS_FAILED_INCORRECT_PARAMETERS = 4;

  /**
   * This feature is not supported on this platform.
   */
  QSH_BLE_EXT_ADV_STATUS_FAILED_FEATURE_UNSUPPORTED = 5;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:30:23 GMT hyungchul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  /**
   * Advetising stopped as BT SOC is turned off.
   */
  QSH_BLE_EXT_ADV_STATUS_FAILED_BT_TURNED_OFF = 6;
}

message qsh_ble_ext_evt_adv_started_event
{
  /**
   * Status of the operation, value of qsh_ble_ext_adv_status.
   */
  required uint32 status = 1;
}

message qsh_ble_ext_evt_adv_stopped_event
{
  /**
   * Status of the operation, value of qsh_ble_ext_adv_status.
   */
  required uint32 status = 1;
}

message qsh_ble_ext_evt_adv_random_address_set_event
{
  /**
   * Status of the operation, value of qsh_ble_ext_adv_status.
   */
  required uint32 status = 1;
}

// BLE scanning:

/**
 * Except SCAN_MODE_OPPORTUNISTIC, all other scan modes' scan_interval_ms,
scan_window_ms and priority
 * are reconfigurable by json conf file. The priorities of enabled scan modes in
the json conf file
 * should be >= 1.
 * The scan mode whose priority is -1 will be disabled. When re-configuring scan
modes by using
 * json conf file, scan modes except QSH_BLE_EXT_SCAN_MODE_OEM_* shall not be
disabled.
 */
enum qsh_ble_ext_scan_mode_constants
{
  option (nanopb_enumopt).long_names = false;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

```
  /**
   * A special Bluetooth LE scan mode. Applications using this scan mode will
passively
   * listen for other scan results without starting BLE scans themselves.
   */
  QSH_BLE_EXT_SCAN_MODE_OPPORTUNISTIC = -1;

  /**
   * Perform Bluetooth LE scan in low power mode. This is the default scan mode as
it consumes the
   * least power.
   * Default scan_interval_ms/scan_window_ms/priority for this scan mode is
1400/140/1.
   */
  QSH_BLE_EXT_SCAN_MODE_LOW_POWER = 0;

  /**
   * Perform Bluetooth LE scan in balanced power mode. Scan results are returned
at a rate that
   * provides a good trade-off between scan frequency and power consumption.
   * Default scan_interval_ms/scan_window_ms/priority for this scan mode is
730/183/2.
   */
  QSH_BLE_EXT_SCAN_MODE_BALANCED = 1;

  /**
   * Deprecated. It is same as SCAN_MODE_BALANCED, use SCAN_MODE_BALANCED
   * instead of this mode.
   */
  QSH_BLE_EXT_SCAN_MODE_AMBIENT_DISCOVERY = 2;

  /**
   * Scan using highest duty cycle.
   * Default scan_interval_ms/scan_window_ms/priority for this scan mode is
100/100/3.
   */
  QSH_BLE_EXT_SCAN_MODE_LOW_LATENCY = 3;

  /**
   * Below scan modes are not enabled by default, configure
scan_interval_ms/scan_window_ms/priority
   * by json conf file to enable them if needed.
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.woo@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
   */
  QSH_BLE_EXT_SCAN_MODE_OEM_1 = 51;
  QSH_BLE_EXT_SCAN_MODE_OEM_2 = 52;
  QSH_BLE_EXT_SCAN_MODE_OEM_3 = 53;
  QSH_BLE_EXT_SCAN_MODE_OEM_4 = 54;
  QSH_BLE_EXT_SCAN_MODE_OEM_5 = 55;
  QSH_BLE_EXT_SCAN_MODE_OEM_6 = 56;
}

message qsh_ble_ext_scan_settings
{
  /**
   * scan mode for Bluetooth LE scan.
   *
   * The scan mode can be one of {QSH_BLE_EXT_SCAN_MODE_OPPORTUNISTIC},
   * {QSH_BLE_EXT_SCAN_MODE_LOW_POWER}, {QSH_BLE_EXT_SCAN_MODE_BALANCED},
   * {QSH_BLE_EXT_SCAN_MODE_LOW_LATENCY} and
{QSH_BLE_EXT_SCAN_MODE_AMBIENT_DISCOVERY}.
   */
  required sint32 scan_mode = 1;

  /**
   * Batch scan parameter for time threshold. Unit is millisecond.
   * Batch scanning is not supported currently
   */
  optional uint32 max_latency = 2;

  /**
   * Default value is -128
   */
  optional sint32 high_rssi_threshold = 3;
}

message qsh_ble_ext_scan_filter
{
  /**
   * filter on service data. For any bit in the service_data_mask, set it to 1
   * if it needs to match the one in service data, otherwise set it to 0 to
   * ignore that bit.
   * If sensor client uses below service data filter, service_data_uuid will be
   * mandatory. Besides, if service_data_mask is not null, it must have the
   * same length as the service_data.
   */
  optional qsh_ble_ext_uuid service_data_uuid = 1;
  optional bytes service_data = 2;
  optional bytes service_data_mask = 3;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  /**
   * filter on manufacturer data. For any bit in the manufacturer_data_mask, set
it to 1
   * if it needs to match the one in manufacturer data, otherwise set it to 0 to
   * ignore that bit.
   * If sensor client uses below manufacturer data filter, manufacturer_id will be
   * mandatory. Besides, if manufacturer_data_mask is not null, it must have the
   * same length as the manufacturer_data.
   */
  optional uint32 manufacturer_id = 4;
  optional bytes manufacturer_data = 5;
  optional bytes manufacturer_data_mask = 6;

  /**
   * filter on device name.
   */
  optional string device_name = 7;

  /**
   * filter on service uuid. The service_uuid_mask is the bit mask for the
   * {service_uuid}. Set any bit in the mask to 1 to indicate a match is needed
for the
   * bit in service_uuid, and 0 to ignore that bit. service_uuid_mask is optional.
   * If sensor client uses below service uuid filter, service_uuid will be
mandatory.
   * service_uuid_mask is optional.
   */
  optional qsh_ble_ext_uuid service_uuid = 8;
  optional qsh_ble_ext_uuid service_uuid_mask = 9;

  /**
   * filter on service solicitation uuid. The service_solicitation_uuid_mask is
the bit mask for the
   * {service_solicitation_uuid}. Set any bit in the mask to 1 to indicate a match
is needed for the
   * bit in service_solicitation_uuid, and 0 to ignore that bit.
service_solicitation_uuid_mask is optional.
   * If sensor client uses below service solicitation uuid filter,
service_solicitation_uuid will be mandatory.
   * service_solicitation_uuid_mask is optional.
   */
  optional qsh_ble_ext_uuid service_solicitation_uuid = 10;
  optional qsh_ble_ext_uuid service_solicitation_uuid_mask = 11;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:30:23 GMT hyungchul.won@nge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_ble_ext_req_scan_start_scan
{
  /**
   * settings for the scan.
   */
  required qsh_ble_ext_scan_settings scan_settings = 1;

  /**
   * list of ScanFilters for filtering advertisements.
   */
  repeated qsh_ble_ext_scan_filter scan_filters = 2;

  /**
   * when set to true, QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_RAW_HCI_EVENT
   * will be used to deliver advertising event,
   * instead of using QSH_BLE_EXT_MSGID_QSH_BLE_EXT_EVT_SCAN_RESULT_EVENT.
   */
  optional bool report_raw_event = 3 [default = false];
}

enum qsh_ble_ext_scan_status
{
  option (nanopb_enumopt).long_names = false;

  /**
   * The requested operation was successful.
   */
  QSH_BLE_EXT_SCAN_STATUS_SUCCESS = 0;

  /**
   * Fails to start scan as BLE scan within the same data stream is already
started.
   */
  QSH_BLE_EXT_SCAN_STATUS_FAILED_ALREADY_STARTED = 1;

  /**
   * Failed to register scanner because no scan instance is available.
   */
  QSH_BLE_EXT_SCAN_STATUS_FAILED_TOO_MANY_SCANNERS = 2;

  /**
   * Operation failed due to an internal error.
   */
  QSH_BLE_EXT_SCAN_STATUS_FAILED_INTERNAL_ERROR = 3;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  /**
   * Fails to start scan as the parameters set by sensor client are incorrect.
   */
  QSH_BLE_EXT_SCAN_STATUS_FAILED_INCORRECT_PARAMETERS = 4;

  /**
   * Fails to start scan as it is out of hardware resources.
   */
  QSH_BLE_EXT_SCAN_STATUS_FAILED_OUT_OF_HARDWARE_RESOURCES = 5;

  /**
   * Scan stopped as BT SOC is turned off.
   */
  QSH_BLE_EXT_SCAN_STATUS_FAILED_BT_TURNED_OFF = 6;
}


message qsh_ble_ext_evt_scan_started_event
{
  /**
   * Status of the operation, value of qsh_ble_ext_scan_status.
   */
  required uint32 status = 1;
}

message qsh_ble_ext_evt_scan_stopped_event
{
  /**
   * Status of the operation, value of qsh_ble_ext_scan_status.
   */
  required uint32 status = 1;
}

message qsh_ble_ext_scan_record
{
  /**
   * Raw bytes of scan record. Sensor client can parse it to extract
   * the needed data if the data doesn't belong to following fields.
   */
  optional bytes raw_adv_data = 1;

  /**
   * List of service data.
   */
  repeated qsh_ble_ext_service_data service_data_s = 2;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  /**
   * List of manufacturer data.
   */
  repeated qsh_ble_ext_manufacturer_data manufacturer_data_s = 3;

  /**
   * Remote device name.
   */
  optional string device_name = 4;

  /**
   * List of service uuid.
   */
  repeated qsh_ble_ext_uuid service_uuid_s = 5;

  /**
   * List of service solicitation uuid.
   */
  repeated qsh_ble_ext_uuid service_solicitation_uuid_s = 6;
}

message qsh_ble_ext_evt_scan_result_event
{
  /**
   * The scan record.
   */
  optional qsh_ble_ext_scan_record scan_record = 1;

  /**
   * Remote device address type.
   */
  required uint32 device_address_type = 2;

  /**
   * Remote device address.
   */
  required bytes device_address = 3;

  /**
   * The received signal strength in dBm.
   * The valid range is [-127, 20].
   */
  required sint32 rssi = 4;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
/**
 * The remote transmit power in dBm.
 * Valid range is [-127, 20].
 * 0x7F indicates that TX power is not available.
 */
required sint32 tx_power = 5;

/**
 * the system time tick when the scan result was observed.
 * value is sns_get_system_time()
 */
required fixed64 time_stamp = 6;

/**
* Adv event type
*/
required uint32 event_type = 7;

/**
* Indicates primary phy used to send advertsing pdu
*/
required uint32 primary_phy = 8;

/**
* Indicates secondary phy used to send advertsing pdu
*/
required uint32 secondary_phy = 9;

/**
* Periodic adv interval.
* Set to 0 if no periodic advertisment is present
*/
required uint32 pa_interval = 10;

/**
* Advertising SID
*/
required uint32 adv_sid = 11;

/**
* Direct address and address type of Advertiser
* These fields are valid only for directed advertisments event_type
* second bit is 1
*/
required uint32 direct_address_type = 12;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yoon@lge.com

```
  required bytes direct_address = 13;
}

message qsh_ble_ext_evt_raw_hci_event
{
  /**
  * This field contains the whole HCI event. The first byte is 0x04 which
  * indicates that packet type is HCI event. Second byte is Event Code.
  */
  required bytes hci_event = 1;
}
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:36:23 GMT
hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

7 QSH Location API

QSH Location API is available at adsp_proc/qsh/qsh_location/pb/qsh_location.proto in the ADSP build.

```
// @file qsh_location.proto
//
// Defines standard message types for LOCATION
//
// Copyright (c) 2020-2021 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.

syntax = "proto2";
import "nanopb.proto";
import "sns_std.proto";

// Location Attribute Requirements:
// SNS_STD_SENSOR_ATTRID_TYPE: "location"
// SNS_STD_SENSOR_ATTRID_AVAILABLE: true
// SNS_STD_SENSOR_ATTRID_API: "qsh_location.proto"
// SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_STREAMING

enum qsh_location_msgid
{
  option (nanopb_enumopt).long_names = false;

  // Uses message: qsh_location_open
  // Direction: From client to sensor
  // Purpose: Attempts to open the location driver with required version
  // Expects QSH_LOCATION_MSGID_QSH_LOCATION_ACK from the sensor
  QSH_LOCATION_MSGID_QSH_LOCATION_OPEN = 512;

  // Does not use any message body.
  // Direction: From client to sensor
  // Purpose: Attempts to close the location driver
  // Expects QSH_LOCATION_MSGID_QSH_LOCATION_ACK from the sensor
  QSH_LOCATION_MSGID_QSH_LOCATION_CLOSE = 513;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyunchul.won@qc.com

```
  // Uses message: qsh_location_update
  // Direction: From client to sensor
  // Purpose: Starts/stops tracking requiring positions or measurements with
  // the specified rate
  // On receiving this request for start=1, the tracking session is started and the
  // sensor will then send either QSH_LOCATION_MSGID_QSH_LOCATION_POSITION_EVENT or
  // QSH_LOCATION_MSGID_QSH_LOCATION_MEAS_AND_CLK_EVENT based on the location_request
  // at the specified interval until the session is stopped by the same request with
  // start=0
  // Expects QSH_LOCATION_MSGID_QSH_LOCATION_ACK from the sensor
  QSH_LOCATION_MSGID_QSH_LOCATION_UPDATE = 514;

  // Uses message: qsh_location_ack
  // Direction: From sensor to client
  // Purpose: ACK/NACK for all commands
  // Client uses this as an indication that the command was successful/completed
  // and it can send subsequent commands. Some information might be sent to the
  // client as well for some commands, e.g. capabilities of the sensor
  QSH_LOCATION_MSGID_QSH_LOCATION_ACK = 768;

  // Uses message: qsh_location_position_event
  // Direction: From sensor to client
  // Purpose: Reports information related to position fixes to the client
  QSH_LOCATION_MSGID_QSH_LOCATION_POSITION_EVENT = 1024;

  // Uses message: qsh_location_meas_and_clk_event
  // Direction: From sensor to client
  // Purpose: Reports information related to measurements and clock to the client
  QSH_LOCATION_MSGID_QSH_LOCATION_MEAS_AND_CLK_EVENT = 1025;
}

message qsh_location_open
{
  // requested version number. The sensor must support a version with the same
  // major version as the one requested
  required uint32 version = 1;
}
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 03:36:23 GMT hyungchul.won@zte.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
enum qsh_location_request
{
  option (nanopb_enumopt).long_names = false;

  // position requests
  QSH_LOCATION_POSITION_REQUEST = 1;

  // measurement and clock requests
  QSH_LOCATION_MEAS_AND_CLK_REQUEST = 2;
}

message qsh_location_update
{
  // start/stop indicator, true to start the session, false to stop the session
  required bool start = 1;

  // indicate if either positions or measurements/clock are requested
  required qsh_location_request location_request = 2;

  // the required interval between positions or measurements in ms
  required uint32 interval = 3;
}

enum qsh_location_capabilities
{
  option (nanopb_enumopt).long_names = false;

  // The driver supports position requests
  QSH_LOCATION_CAPABILITIES_POSITION = 1;

  // The driver supports measurement and clock requests
  QSH_LOCATION_CAPABILITIES_MEAS_AND_CLK = 2;
}

message qsh_location_ack
{
  // The command/report was succesful or not (true/false)
  // If trying to open the driver when it is already open
  // this will be false
  required bool error = 1;

  // The command/report for which this ACK is sent
  required qsh_location_msgid command = 2;

  // driver's version number. The driver must support a version with the same
  // major version as the one requested
  // filled in only for Open command
  optional uint32 version = 3;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 09:36:23 GMT hyungchul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // driver's capabilities
  // filled in only for Open command, see qsh_location_capabilities
  // bitwise OR of qsh_location_capabilities bits
  optional uint32 capabilities = 4;
}

message qsh_location_position_event
{
  // UTC timestamp for location fix in milliseconds since January 1, 1970
  required uint64 timestamp = 1;

  // Fixed point latitude in degrees times 10^7
  optional int32 latitude = 2;

  // Fixed point longitude in degrees times 10^7
  optional int32 longitude = 3;

  // Fixed point altitude in meters above the WGS 84 reference ellipsoid
  optional float altitude = 4;

  // Horizontal accuracy in meters such that a circle with a radius
  // of length 'accuracy' from the latitude and longitude has a 68%
  // probability of including the true location.
  optional float horizontal_accuracy = 5;

  // Vertical accuracy in meters such that a range of
  // 2 * altitude_accuracy centered around altitude has a 68%
  // probability of including the true altitude.
  optional float vertical_accuracy = 6;

  // Horizontal speed in meters/second
  optional float speed = 7;

  // Speed accuracy in meters per second such that a range of
  // 2 * speed_accuracy centered around speed has a 68% probability of
  // including the true speed.
  optional float speed_accuracy = 8;

  // Clockwise angle between north and current heading, in degrees; range
  // [0, 360)
  optional float bearing = 9;

  // Bearing accuracy in degrees such that a range of
  // 2 * bearing_accuracy centered around bearing has a 68% probability of
  // including the true bearing.
  optional float bearing_accuracy = 10;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@rse.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // Conformity index. Indicates how well the various input data considered for
  // navigation solution conform to expectations
  // Range: 0 (least conforming) to 1 (most conforming)
  optional float conformity_index = 11;
}

enum qsh_location_constellation_type
{
  option (nanopb_enumopt).long_names = false;

  QSH_LOCATION_CONSTELLATION_TYPE_UNKNOWN    = 0;
  QSH_LOCATION_CONSTELLATION_TYPE_GPS        = 1;
  QSH_LOCATION_CONSTELLATION_TYPE_SBAS       = 2;
  QSH_LOCATION_CONSTELLATION_TYPE_GLONASS    = 3;
  QSH_LOCATION_CONSTELLATION_TYPE_QZSS       = 4;
  QSH_LOCATION_CONSTELLATION_TYPE_BEIDOU     = 5;
  QSH_LOCATION_CONSTELLATION_TYPE_GALILEO    = 6;
  QSH_LOCATION_CONSTELLATION_TYPE_NAVIC      = 7;
}

enum qsh_location_measurement_state
{
  option (nanopb_enumopt).long_names = false;

  // Flags indicating the GNSS measurement state
  QSH_LOCATION_MEASUREMENT_STATE_UNKNOWN                    = 0;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_CODE_LOCK            = 0x00000001;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_BIT_SYNC             = 0x00000002;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_SUBFRAME_SYNC        = 0x00000004;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_TOW_DECODED          = 0x00000008;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_MSEC_AMBIGUOUS       = 0x00000010;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_SYMBOL_SYNC          = 0x00000020;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_GLO_STRING_SYNC      = 0x00000040;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_GLO_TOD_DECODED      = 0x00000080;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_BDS_D2_BIT_SYNC      = 0x00000100;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_BDS_D2_SUBFRAME_SYNC = 0x00000200;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_GAL_E1BC_CODE_LOCK   = 0x00000400;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_GAL_E1C_2ND_CODE_LOCK = 0x00000800;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_GAL_E1B_PAGE_SYNC    = 0x00001000;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_SBAS_SYNC            = 0x00002000;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_TOW_KNOWN            = 0x00004000;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_GLO_TOD_KNOWN        = 0x00008000;
  QSH_LOCATION_MEASUREMENT_STATE_STATE_2ND_CODE_LOCK        = 0x00010000;
}
```

Qualcomm Confidential – May Contain Trade Secrets

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
enum qsh_location_multipath_indicator
{
  option (nanopb_enumopt).long_names = false;

  QSH_LOCATION_MULTIPATH_INDICATOR_UNKNOWN      = 0;
  QSH_LOCATION_MULTIPATH_INDICATOR_PRESENT      = 1;
  QSH_LOCATION_MULTIPATH_INDICATOR_NOT_PRESENT  = 2;
}

message qsh_location_measurement
{
  // Satellite vehicle ID number
  required int32 sv_id = 1;

  // Constellation of the given satellite vehicle
  required qsh_location_constellation_type constellation_type = 2;

  // Hardware time offset from time_ns for this measurement, in nanoseconds
  required int64 time_offset = 3;

  // Satellite sync state flags, see qsh_location_measurement_state
  // bitwise OR of qsh_location_measurement_state bits
  required uint32 state = 4;

  // Received GNSS satellite time at the time of measurement, in nanoseconds
  required int64 received_sv_time = 5;

  // 1-sigma uncertainty of received GNSS satellite time, in nanoseconds
  required int64 received_sv_time_uncertainty = 6;

  // Carrier-to-noise density in dB-Hz, measured at the antenna, in the range of
[0, 63]
  required float c_n0 = 7;

  // Pseudorange rate at the timestamp in meters per second (uncorrected)
  required float pseudorange_rate = 8;

  // 1-sigma uncertainty of pseudorange rate in meters per second
  required float pseudorange_rate_uncertainty = 9;

  // Carrier phase measurement (L1 cycles)
  optional double carrier_phase = 10;

  // 1-sigma uncertainty of the carrier phase measurement
  optional float carrier_phase_uncertainty = 11;

  // Increments when a cycle slip is detected
  optional uint32 cycle_slip_count = 12;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:30:23 GMT hyungchul.yun@line.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // multipath indicator, see qsh_location_multipath_indicator
  optional qsh_location_multipath_indicator multipath_indicator = 13;

  // Signal to noise ratio (dB), power above observed noise at correlators
  optional float snr = 14;

  // Carrier frequency of the signal tracked in Hz.
  // For example, it can be the GPS central frequency for L1 = 1575.45 MHz,
  // or L2 = 1227.60 MHz, L5 = 1176.45 MHz, various GLO channels, etc.
  //
  // For an L1, L5 receiver tracking a satellite on L1 and L5 at the same
  // time, two chreGnssMeasurement structs must be reported for this same
  // satellite, in one of the measurement structs, all the values related to
  // L1 must be filled, and in the other all of the values related to L5
  // must be filled.
  optional float carrier_frequency = 15;
}

message qsh_location_clock
{
  // The GNSS receiver hardware clock value in nanoseconds
  required int64 time = 1;

  // The GNSS receiver hardware clock value uncertainty in nanoseconds
  optional float time_uncertainty = 2;

  // The difference between hardware clock inside GNSS receiver and the
  // estimated GNSS time in nanoseconds; contains bias uncertainty
  optional int64 full_bias = 3;

  // Sub-nanosecond bias, adds to full_bias
  optional float bias = 4;

  // 1-sigma uncertainty associated with the clock's bias in nanoseconds
  optional float bias_uncertainty = 5;

  // The clock's drift in nanoseconds per second
  optional float drift = 6;

  // 1-sigma uncertainty associated with the clock's drift in nanoseconds
  // per second
  optional float drift_uncertainty = 7;

  // While this number stays the same, timeNs should flow continuously
  optional uint32 hw_clock_discontinuity_count = 8;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:30:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_location_meas_and_clk_event
{
  // Number of qsh_location_measurement entries included in the report for this
epoch.
  // Must be in the range [0, 128]. Note that multiple messages might be used to
  // transmit all the measurements for one epoch, please see comment below for
  // measurement_index for an example
  required uint32 number_of_measurements = 1;

  // array of measurements, see qsh_location_measurement
  repeated qsh_location_measurement measurements = 2;

  // clock information, see qsh_location_clock
  required qsh_location_clock clock = 3;

  // indicates the measurement index. This is useful in case there is not enough
  // room in the message to send all the measurements. As an example, if
  // number_of_measurements = 87, but only 30 measurements can be sent in one
  // message measurement_index will be 1 for the first message, 31 for the
  // second, 61 for the third. Note that in this case clock information will
  // be identical for all three messages
  required uint32 measurement_index = 4;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

8 QSH Geofence API

QSH Geofence API is available at adsp_proc/qsh/qsh_geofence/pb/qsh_geofence.proto in the ADSP build.

```
// @file qsh_geofence.proto
//
// Defines standard message types for GEOFENCE
//
// Copyright (c) 2021 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.

syntax = "proto2";
import "nanopb.proto";
import "sns_std.proto";

// Geofence Attribute Requirements:
// SNS_STD_SENSOR_ATTRID_TYPE: "geofence"
// SNS_STD_SENSOR_ATTRID_AVAILABLE: true
// SNS_STD_SENSOR_ATTRID_API: "qsh_geofence.proto"
// SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_ON_CHANGE

enum qsh_geofence_msgid
{
  option (nanopb_enumopt).long_names = false;

  // Uses message: qsh_geofence_open
  // Direction: From client to sensor
  // Purpose: Attempts to open the geofence driver with required version
  // Expects QSH_GEOFENCE_MSGID_QSH_GEOFENCE_ACK from the sensor
  QSH_GEOFENCE_MSGID_QSH_GEOFENCE_OPEN = 512;

  // Does not use any message body.
  // Direction: From client to sensor
  // Purpose: Attempts to close the geofence driver
  // Expects QSH_GEOFENCE_MSGID_QSH_GEOFENCE_ACK from the sensor
  QSH_GEOFENCE_MSGID_QSH_GEOFENCE_CLOSE = 513;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyunchul.won@qti.com

```
  // Uses message: qsh_geofence_update
  // Direction: From client to sensor
  // Purpose: Attempts to add/remove/modify/pause/resume geofences
  // in the geofence driver
  // Expects QSH_GEOFENCE_MSGID_QSH_GEOFENCE_ACK from the sensor
  QSH_GEOFENCE_MSGID_QSH_GEOFENCE_UPDATE = 514;

  // Uses message: qsh_geofence_ack
  // Direction: From sensor to client
  // Purpose: ACK/NACK for all geofence commands
  // Client uses this as an indication that the geofence command was
successful/completed
  // and it can send subsequent commands. Some information might be sent to the
  // client as well for some commands, e.g. the geofence(s) ID(s)
  QSH_GEOFENCE_MSGID_QSH_GEOFENCE_ACK = 768;

  // Uses message: qsh_geofence_breach_event
  // Direction: From sensor to client
  // Purpose: Reports to the client when any number of geofences have a state
change,
  // i.e. whenever there is a geofence breach
  QSH_GEOFENCE_MSGID_QSH_GEOFENCE_BREACH_EVENT = 1024;
}

message qsh_geofence_open
{
  // requested version number. The sensor must support a version with the same
  // major version as the one requested
  required uint32 version = 1;
}

enum qsh_geofence_subcommand
{
  option (nanopb_enumopt).long_names = false;

  // Add geofence(s)
  QSH_GEOFENCE_ADD = 1;

  // Remove geofence(s)
  QSH_GEOFENCE_REMOVE = 2;

  // Modify geofence(s)
  QSH_GEOFENCE_MODIFY = 3;

  // Pause geofence(s)
  QSH_GEOFENCE_PAUSE = 4;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // Resume geofence(s)
  QSH_GEOFENCE_RESUME = 5;
}

enum qsh_geofence_breach_type
{
  option (nanopb_enumopt).long_names = false;

  QSH_GEOFENCE_BREACH_ENTER            = 0;
  QSH_GEOFENCE_BREACH_EXIT             = 1;
  QSH_GEOFENCE_BREACH_DWELL_IN         = 2;
  QSH_GEOFENCE_BREACH_DWELL_OUT        = 3;
}

enum qsh_geofence_breach_type_mask
{
  option (nanopb_enumopt).long_names = false;

  QSH_GEOFENCE_BREACH_ENTER_BIT        = 0x00000001;
  QSH_GEOFENCE_BREACH_EXIT_BIT         = 0x00000002;
  QSH_GEOFENCE_BREACH_DWELL_IN_BIT     = 0x00000004;
  QSH_GEOFENCE_BREACH_DWELL_OUT_BIT    = 0x00000008;
}

message qsh_geofence_specification
{
  // geofence ID
  optional uint32 id = 1;

  // Bitwise OR of qsh_geofence_breach_type_mask bits
  // Either QSH_GEOFENCE_BREACH_ENTER_BIT or QSH_GEOFENCE_BREACH_EXIT_BIT
  // MUST be on, therefore the possible valueas are:
  // 1  = QSH_GEOFENCE_BREACH_ENTER_BIT
  // 2  = QSH_GEOFENCE_BREACH_EXIT_BIT
  // 5  = QSH_GEOFENCE_BREACH_ENTER_BIT | QSH_GEOFENCE_BREACH_DWELL_IN_BIT
  // 10 = QSH_GEOFENCE_BREACH_EXIT_BIT | QSH_GEOFENCE_BREACH_DWELL_OUT_BIT
  optional uint32 breach_type_mask = 2;

  // Responsiveness in milliseconds
  // Specifies the rate of detection for a Geofence breach.
  // This can impact the time lag between the actual breach event and
  // when it is reported. This parameter has power implications
  // and must be fine-tuned to optimize power savings
  // The gap between the actual breach and
  // the time it is reported depends on the user setting. The power implication
  // is inversely proportional to the responsiveness value set by the user.
  // The higher the responsiveness value, the lower the power implications, and
vice-versa.
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hjunsoul.kim@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  optional uint32 responsiveness = 3;

  // Dwell time in seconds
  optional uint32 dwell_time = 4;

  // Fixed point latitude in degrees times 10^7
  optional int32 latitude = 5;

  // Fixed point longitude in degrees times 10^7
  optional int32 longitude = 6;

  // Radius in meters
  optional float radius = 7;
}

message qsh_geofence_update
{
  // indicate if this is add/remove/modify/pause/resume geofences
  required qsh_geofence_subcommand geofence_subcommand = 1;

  // array of geofence specifications, see qsh_geofence_specification
  repeated qsh_geofence_specification specs = 2;
}

enum  qsh_geofence_error_type
{
  option (nanopb_enumopt).long_names = false;

  // Success
  QSH_GEOFENCE_ERROR_SUCCESS                = 0;

  // General failure
  QSH_GEOFENCE_ERROR_GENERAL_FAILURE        = 1;

  // Callback is missing
  QSH_GEOFENCE_ERROR_CALLBACK_MISSING       = 2;

  // Invalid parameter
  QSH_GEOFENCE_ERROR_INVALID_PARAMETER      = 3;

  // ID already exists
  QSH_GEOFENCE_ERROR_ID_EXISTS              = 4;

  // ID is unknown
  QSH_GEOFENCE_ERROR_ID_UNKNOWN             = 5;

  // Already started
  QSH_GEOFENCE_ERROR_ALREADY_STARTED        = 6;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
  // Not initialized
  QSH_GEOFENCE_ERROR_NOT_INITIALIZED        = 7;

  // Maximum number of geofences reached
  QSH_GEOFENCE_ERROR_GEOFENCES_AT_MAX       = 8;
}

message qsh_geofence_ack
{
  // The command/report was succesful or not (true/false)
  required bool error = 1;

  // The command/report for which this ACK is sent
  required qsh_geofence_msgid command = 2;

  // driver's version number. The driver must support a version with the same
  // major version as the one requested
  // filled in only for Open command
  optional uint32 version = 3;

  // The subcommand for which this ACK is sent in case
  // command = QSH_GEOFENCE_MSGID_QSH_GEOFENCE_UPDATE
  optional qsh_geofence_subcommand subcommand = 4;

  // array of geofence IDs
  // maximum number of entries in the array = 30
  repeated uint32 ids = 5;

  // array of of qsh_geofence_error_type associated with the request
  // maximum number of entries in the array = 30
  // the number of ids is the same as the number of errs
  repeated uint32 errs = 6;
}

enum qsh_geofence_flags_type
{
  option (nanopb_enumopt).long_names = false;

  // Location has a valid latitude and longitude
  QSH_GEOFENCE_HAS_LAT_LONG_BIT               = 0x00000001;

  // Location has a valid altitude
  QSH_GEOFENCE_HAS_ALTITUDE_BIT               = 0x00000002;

  // Location has a valid speed
  QSH_GEOFENCE_HAS_SPEED_BIT                  = 0x00000004;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.kim@lge.com

```
  // Location has a valid bearing
  QSH_GEOFENCE_HAS_BEARING_BIT                = 0x00000008;

  // Location has valid accuracy
  QSH_GEOFENCE_HAS_ACCURACY_BIT               = 0x00000010;
}

message qsh_geofence_breach_event
{
  // array of geofence IDs
  // maximum number of entries in the array = 30
  repeated uint32 ids = 1;

  // UTC timestamp for a location fix; milliseconds since Jan. 1, 1970
  required uint64 location_timestamp = 2;

  // Fixed point latitude in degrees times 10^7
  // This is the latitude where the breach occurred
  required int32 latitude = 3;

  // Fixed point longitude in degrees times 10^7
  // This is the longitude where the breach occurred
  required int32 longitude = 4;

  // Altitude in meters above the WGS 84 reference ellipsoid
  required float altitude = 5;

  // Speed in meters per second
  required float speed = 6;

  // Bearing in degrees; range: 0 to 360
  required float bearing = 7;

  // Accuracy in meters
  required float accuracy = 8;

  // Bitwise OR of qsh_geofence_flags_type
  required uint32 flags = 9;

  // Type of breach (see qsh_geofence_breach_type)
  required uint32 breach_type = 10;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yun@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

9 QSH Wi-Fi API

QSH Wi-Fi API is available at adsp_proc/qsh/qsh_wifi/pb/qsh_wifi.proto in the ADSP build.

```
// @file qsh_wifi.proto
//
// Defines standard message types for WIFI
//
// Copyright (c) 2020 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.

/*
      Not a Contribution.
 * Copyright (C) 2016 The Android Open Source Project
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */

syntax = "proto2";
import "nanopb.proto";
import "sns_std.proto";

/**
 * WIFI Attribute Requirements:
 * SNS_STD_SENSOR_ATTRID_TYPE: "wifi_svc"
 * SNS_STD_SENSOR_ATTRID_AVAILABLE: true
 * SNS_STD_SENSOR_ATTRID_API: "qsh_wifi.proto"
 * SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_STREAMING
 */
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:30:23 GMT hyungchul.won@lge.com

```
/**
 * Listing the general guidelines for Command and Events.
 *
 * Here onwards sensor driver (running on SLPI/DSP) will be
 * referred as sensor and sensor client (on apps or slpi)
 * will be referred as client.
 *
 * Each Command (CMD) direction is from client --> sensor.
 * On reception of each CMD, sensor will send ACK to client.
 * When result is available with sensor, it will send asynchronously to client.
 * So Each Event (EVT) direction is from sensor --> client.
 *
 */
enum qsh_wifi_msgid
{
    option (nanopb_enumopt).long_names = false;

    // Uses message : qsh_wifi_cmd_open
    // Direction    : From client to sensor
    // Purpose      : Initial handshake with sensor to get sensor supported
    //                capabilities and version.
    //                Note : 1.version of sensor is mainly used for
    //                information purpose and to assist in debugging.
    //                2.sensor will be always backward compatible.
    // Expectation  : QSH_WIFI_MSGID_QSH_WIFI_CMD_ACK
    // Guideline    : No seperate close cmd is needed because it will happen
    //                when the client close the QSH data stream (for slpi client)
    //                or close the QMI handle (for apps client).
    QSH_WIFI_MSGID_QSH_WIFI_CMD_OPEN = 512;

    // Uses message : qsh_wifi_cmd_scan_req
    // Direction    : From client to sensor
    // Purpose      : Issue a scan request to sensor
    // Expectation  : If the sensor accepts the scan request then, the sensor shall
    //                send ack via QSH_WIFI_MSGID_QSH_WIFI_CMD_ACK with success
    //                status. Later when scan results are available with sensor,
    //                it will send via QSH_WIFI_MSGID_QSH_WIFI_EVT_SCAN_RESULTS.
    //                Note: 1.If the sensor does not accept the scan request then,
    //                the sensor shall send QSH_WIFI_CMD_ACK with failure status.
    //                For e.g.- If the sensor is servicing other scan request then,
    //                the sensor shall send QSH_WIFI_MSGID_QSH_WIFI_CMD_ACK with
    //                QSH_WIFI_ERROR_BUSY status.
    // Guideline    : client shall check the status return by sensor in scan event
    //                (QSH_WIFI_MSGID_QSH_WIFI_EVT_SCAN_RESULTS). There is a chance
    //                that scan request may not succeed even though the command was
    //                accepted. The sensor shall send final status of request
    //                  accordingly in scan event.
    QSH_WIFI_MSGID_QSH_WIFI_CMD_SCAN_REQ = 513;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // Uses message : qsh_wifi_cmd_monitor_req
    // Direction    : From client to sensor
    // Purpose      : Configure monitor mode scan in sensor
    // Expectation :  If the sensor accept the monitor request then, the sensor
    //                shall send QSH_WIFI_MSGID_QSH_WIFI_CMD_ACK with success
    //                status. Later when the sensor have monitored scan results
    //                then, the sensor shall send scan results via
    //                QSH_WIFI_MSGID_QSH_WIFI_EVT_SCAN_RESULTS.
    //                If the sensor does not accept the monitor request then,
    //                the sensor shall send QSH_WIFI_CMD_ACK with a failure status.
    // Guideline    : client shall check the status return by sensor in
    //                ACK (QSH_WIFI_MSGID_QSH_WIFI_CMD_ACK).It is responsibility
    //                of sensor to holding this request unless client stop
    //                monitor mode request. client shall stop monitor mode before
    //                closing the sensor stream.
    //                In case of wlan chip SSR or wlan UI off scenario, sensor
    //                reregister the monitor mode without a client intervention.
    //                During slpi SSR scenario, apps client will be notfied
    //                using qmi error callbacks. Hence apps clients need to open
    //                and re-register monitor mode request.
    QSH_WIFI_MSGID_QSH_WIFI_CMD_MONITOR_REQ = 514;

    // Uses message : qsh_wifi_cmd_ranging_req
    // Direction    : From client to sensor
    // Purpose      : Issue a ranging request to sensor
    // Expectation  : If the sensor accepts the ranging request then, the sensor shall
    //                send ack via QSH_WIFI_MSGID_QSH_WIFI_CMD_ACK with success
    //                status. Later when ranging results are available with sensor,
    //                it will send via
QSH_WIFI_MSGID_QSH_WIFI_EVT_RANGING_RESULTS_RESULTS.
    //                Note: 1.If the sensor does not accept the ranging request then,
    //                the sensor shall send QSH_WIFI_CMD_ACK with failure status.
    //                For e.g.- If the sensor is servicing other ranging request then,
    //                the sensor shall send QSH_WIFI_MSGID_QSH_WIFI_CMD_ACK with
    //                QSH_WIFI_ERROR_BUSY status.
    // Guideline    : client shall check the status return by sensor in ranging event
    //                (QSH_WIFI_MSGID_QSH_WIFI_EVT_SCAN_RESULTS). There is a chance
    //                that ranging request may not succeed even though the command was
    //                accepted. The sensor shall send final status of request
    //                  accordingly in ranging event.
    QSH_WIFI_MSGID_QSH_WIFI_CMD_RANGING_REQ = 515;

    // Uses message : qsh_wifi_cmd_ack
    // Direction    : From sensor to client
    // Purpose      : Provide Immediate response for issued commands.
    //                client uses this as an indication that the command
    //                was successful accepted/rejected by wifi sensor
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 09:30:23 GMT hyungchul.kim@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    //                to take sub sequent action. This might carry
    //                information e.g. capabilities/version of the sensor.
    QSH_WIFI_MSGID_QSH_WIFI_CMD_ACK = 768;

    // Uses message : qsh_wifi_evt_scan_result
    // Direction    : From sensor to client
    // Purpose      : Provide scan results to client who issued
    //                request for same.
    QSH_WIFI_MSGID_QSH_WIFI_EVT_SCAN_RESULTS = 769;

    // Uses message : qsh_wifi_evt_ranging_result
    // Direction    : From sensor to client
    // Purpose      : Provide ranging results to client who issued
    //                request for same.
    QSH_WIFI_MSGID_QSH_WIFI_EVT_RANGING_RESULTS = 770;
}

/**
 * A generic listing of error codes for use in qsh WiFi sensor driver.
 * In general, Feature specific error codes can be added to this enum.
 */
enum qsh_wifi_error
{
    option (nanopb_enumopt).long_names = false;

    // No error occurred
    QSH_WIFI_ERROR_NONE = 0;

    // An unspecified failure occurred
    QSH_WIFI_ERROR = 1;

    // One or more supplied arguments are invalid
    QSH_WIFI_ERROR_INVALID_ARGS = 2;

    // Unable to satisfy request because the system is busy
    QSH_WIFI_ERROR_BUSY = 3;

    // Unable to allocate memory
    QSH_WIFI_ERROR_NO_MEMORY = 4;

    // The requested feature is not supported
    QSH_WIFI_ERROR_NOT_SUPPORTED = 5;

    // A timeout occurred while processing the request
    QSH_WIFI_ERROR_TIMEOUT = 6;

    // The relevant capability is disabled There may be may reasons
    // listing below few of them
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungsul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // 1) User disabled WLAN  OFF using UI in settings
    // 2) WLAN chip is under SSR or PDR
    // 3) All wlan interface are down.
    QSH_WIFI_ERROR_INTF_DISABLED = 7;
}

/**
 * These enum help client to seek what are the features
 * supported by the WiFi sensor, so client can send request
 * based on the available features.
 */
enum qsh_wifi_capabilities
{
    option (nanopb_enumopt).long_names = false;

    // No WiFi APIs are supported
    QSH_WIFI_CAP_NONE = 0x00000000;

    // Monitor mode aka Listening to scan results is supported
    QSH_WIFI_CAP_SCAN_MONITORING = 0x00000001;

    // WiFi scans based on scan request supported
    QSH_WIFI_CAP_SCAN_REQUEST_BASED = 0x00000002;

    // Requesting RTT ranging is supported
    QSH_WIFI_CAP_RANGING_SCAN = 0x00000004;
}

/**
 * Identifies a WiFi frequency band
 */
enum qsh_wifi_band_mask
{
    option (nanopb_enumopt).long_names = false;

    // Identify Wifi 2.4Ghz band
    QSH_WIFI_BAND_MASK_2P4_GHZ = 0x1;

    // Identify Wifi 5Ghz band
    QSH_WIFI_BAND_MASK_5_GHZ   = 0x2;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
/**
 * Identifies which radio chain was used to discover an AP.
 * The underlying hardware does not necessarily support
 * more than one radio chain.
 */
enum qsh_wifi_radio_chain_index
{
    option (nanopb_enumopt).long_names = false;

    // Indicate Chain unknown
    QSH_WIFI_RADIO_CHAIN_INDEX_UNKNOWN = 0;

    // Indicate Radio Chain 0
    QSH_WIFI_RADIO_CHAIN_INDEX_0 = 1;

    // Indicate Radio Chain 1
    QSH_WIFI_RADIO_CHAIN_INDEX_1 = 2;
}

/**
 * Indicates the type of scan requested or performed
 */
enum qsh_wifi_scan_type
{
    option (nanopb_enumopt).long_names = false;

    // Perform a purely active scan using probe requests. Do not scan channels
    // restricted to use via Dynamic Frequency Selection (DFS) only.
    QSH_WIFI_SCAN_TYPE_ACTIVE = 0;

    // Perform an active scan on unrestricted channels, and also perform a
    // passive scan on channels that are restricted to use via Dynamic
    // Frequency Selection (DFS), e.g. the U-NIII bands 5250-5350MHz and
    // 5470-5725MHz in the USA as mandated by FCC regulation.
    QSH_WIFI_SCAN_TYPE_ACTIVE_AND_PASSIVE_DFS = 1;

    // Perform a passive scan, only listening for beacons.
    QSH_WIFI_SCAN_TYPE_PASSIVE = 2;
}

/**
 * Identifies the authentication methods supported by an AP.
 */
enum qsh_wifi_security
{
    option (nanopb_enumopt).long_names = false;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:22 GMT hyungsul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // Mode is unknown
    QSH_WIFI_SECURITY_MODE_UNKNOWN = 0;

    // No auth/security
    QSH_WIFI_SECURITY_MODE_OPEN = 1;

    // WEP mode
    QSH_WIFI_SECURITY_MODE_WEP = 2;

    // WPA-PSK or WPA2-PSK
    QSH_WIFI_SECURITY_MODE_PSK = 3;

    // Any type of EAPOL
    QSH_WIFI_SECURITY_MODE_EAP = 4;
}

/**
 * Indicates the BSS operating channel width determined from the VHT and/or HT
 * Operation elements. Refer to VHT 8.4.2.161 and HT 7.3.2.57.
 */
enum qsh_wifi_channel_width
{
    option (nanopb_enumopt).long_names = false;

    //Channel Width 20 MHz
    QSH_WIFI_CHANNEL_WIDTH_20       = 0;

    //Channel Width 40 MHz
    QSH_WIFI_CHANNEL_WIDTH_40       = 1;

    //Channel Width 80 MHz
    QSH_WIFI_CHANNEL_WIDTH_80       = 2;

    //Channel Width 160 MHz
    QSH_WIFI_CHANNEL_WIDTH_160      = 3;

    //Channel Width 80+80 MHz
    QSH_WIFI_CHANNEL_WIDTH_80_P_80 = 4;
}

/**
 * Allows a sensor APP to express its preference for how multiple available
 * radio chains should be used when performing scan. This is only a
 * preference from the sensor APP and is not guaranteed to be honored by the
 * WiFi firmware.
 */
enum qsh_wifi_pref_radio_chain
{
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

```
    option (nanopb_enumopt).long_names = false;

    // No preference for radio chain usage
    QSH_WIFI_PREF_RADIO_CHAIN_DEFAULT = 0;

    // Prefer to use available radio chains in a way that minimizes time to
    // complete the scan
    QSH_WIFI_PREF_RADIO_CHAIN_LOW_LATENCY = 1;

    // Prefer to use available radio chains in a way that minimizes total power
    // consumed for the scan
    QSH_WIFI_PREF_RADIO_CHAIN_LOW_POWER = 2;

    // Prefer to use available radio chains in a way that maximizes accuracy of
    // the scan result, e.g. RSSI measurements, total number of found AP's
    QSH_WIFI_PREF_RADIO_CHAIN_HIGH_ACCURACY = 3;
}

/**
 * Characteristics of a scanned device which will be
 * Provided into scan results.
 */
enum qsh_wifi_scan_result_flags
{
    option (nanopb_enumopt).long_names = false;

    // Indicates no feature flag
    QSH_WIFI_SCAN_RESULT_FLAG_NONE = 0x0;

    // Indicate Element ID 61 (HT Operation) is present
    QSH_WIFI_SCAN_RESULT_FLAG_HT_OPS_PRESENT = 0x00000001;

    // Indicate Element ID 192 (VHT Operation) is present
    QSH_WIFI_SCAN_RESULT_FLAG_VHT_OPS_PRESENT = 0x00000002;

    // Indicate Element ID 127 (Extended Capabilities) is present,
    // and bit 70 (Fine Timing Measurement Responder) is set to 1.
    QSH_WIFI_SCAN_RESULT_FLAG_IS_FTM_RESPONDER = 0x00000004;

    // Indicate HT Operation element indicates that a secondary
    // channel is present.
    QSH_WIFI_SCAN_RESULT_FLAG_HAS_SECONDARY_CHANNEL_OFFSET = 0x00000008;

    // Indicate HT Operation element indicates that the secondary
    // channel is below the primary channel.
    QSH_WIFI_SCAN_RESULT_FLAG_SECONDARY_CHANNEL_OFFSET_IS_BELOW = 0x00000010;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:23 GMT hyungchul.kim@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
/**
 * Send open command to the wifi sensor
 */
message qsh_wifi_cmd_open
{
    // Request sensor to provide sensor capabilities like
    // ie what are all feature supported by the sensor.
    // client must check sensor capabilities before issuing
    // the request. Though interface is backward compatible but
    // based on wifi chip capability certain features may be switch off.
    // So it always recommended to check featured supported by sensor.
    required bool get_capability = 1;

    // Request sensor to provide version number of sensor
    optional bool get_version = 2;
}

/**
 * This single message can be used to send all command's immediate
 * response/ack from sensor to sensor clients. If status is generic
 * in nature then it can be set using status(qsh_wifi_error)
 * otherwise specific nature of response can be added via using
 * optional params in this message later.
 */
message qsh_wifi_cmd_ack
{
    // This helps client to decides command/report was successful or not
    required qsh_wifi_error status = 1;

    // This is in immediate response/ack of command for which this ACK is sent
    required qsh_wifi_msgid command = 2;

    // Driver's version number, filled only for Open command
    // First byte denotes Major , Second byte denotes Minor
    // and remaining two bytes will denote sensor build version.
    optional uint32 version = 3;

    // driver's capabilities, filled in only for Open command.
    // see qsh_wifi_capabilities bitwise OR of qsh_wifi_capabilities bits
    optional uint32 capabilities = 4;
}

/**
 * Data structure passed to when qsh wifi based app issue
 * the wifi scan request to qsh wifi sesnor driver.
 */
message qsh_wifi_cmd_scan_req
{
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyunchul.non@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // Set to a value from enum qsh_wifi_scan_type
    required qsh_wifi_scan_type scan_type = 1;

    // Indicates whether the client is willing to tolerate receiving cached
    // results of a previous scan, and if so, the maximum age of the scan that
    // the client will accept. "Age" in this case is defined as the elapsed
    // time between when the most recent scan was completed and the request is
    // received, in milliseconds. If set to 0, no cached results may be
    // provided, and all scan results must come from a "fresh" WiFi scan, i.e.
    // one that completes strictly after this request is received. If more than
    // one scan is cached and meets this age threshold, only the newest scan is
    // provided.
    optional uint32 max_scan_age_ms = 2;

    // Band identifier for which Wifi scan is requested. it is Bitwise OR of
    // qsh_wifi_band_mask.
    required uint32 band = 3;

    // A list of frequencies to scan, given as channel center
    // frequencies in MHz.
    // NOTE: Though we recommend to use band field mainly for scan_req.
    //       If freq_list provided then this will superceed the band value.
    //       ie sensor will only scan the frequency mention in list.
    repeated uint32 freq_list = 4;

    // If ssid_list set then do not restrict scan to any SSIDs.
    // Otherwise, this will list will be used for directed probe requests.
    // Not applicable and ignore when scanType is qsh_wifi_scan_type_passive.
    // SSID will null terminated hence size is 33 bytes as max SSID is 32 bytes
    repeated string ssid_list = 5[(nanopb).max_size = 33];

    // Set to a value from enum qsh_wifi_pref_radio_chain to specify the desired
    // trade-off between power consumption, accuracy, etc.
    optional qsh_wifi_pref_radio_chain radio_chain_pref = 6;
}

/**
 * Provides information about a single access point (AP) detected in a scan.
 */
message qsh_wifi_bss_info
{
    // Basic Service Set Identifier (BSSID), represented in big-endian byte
    // order, such that the first octet of the OUI is accessed in byte index 0.
    required bytes bssid = 1[(nanopb).max_size = 6];
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 09:36:23 GMT hyungchul.won@sec.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
// Number of milliseconds prior to reference Time in the enclosing
// qsh_wifi_evt_scan_result struct when the probe response or beacon frame
// that was used to populate this structure was received.
optional uint32 age_ms = 2;

// Capability Information field sent by the AP (see 802.11 7.3.1.4). This
// field must reflect native byte order and bit ordering, such that
// (capabilityInfo & 1) gives the bit for the ESS subfield.
required uint32 capability_info = 3;

// Service Set Identifier (SSID), a series of 0 to 32 octets identifying
// the access point. Note that this is commonly a human-readable ASCII
// string, but this is not the required encoding per the standard.
optional string ssid = 4[(nanopb).max_size = 33];

// A bitwise OR of qsh_wifi_scan_result_flags_*
optional uint32 flags = 5;

// RSSI (Received Signal Strength Indicator), in dBm. Typically negative
// If multiple radio chains were used to scan this AP, this is a "best
// available" measure that may be a composite of measurements taken across
// the radio chains.
required int32  rssi = 6;

// Operating band, set to a value from enum qsh_wifi_band_mask
required qsh_wifi_band_mask band = 7;

// Indicates the center frequency of the primary 20MHz channel, given in
// MHz. This value is derived from the channel number via the formula:
//
//     primaryChannel (MHz) = CSF + 5 * primaryChannelNumber
//
// Where CSF is the channel starting frequency (in MHz) given by the
// operating class/band (i.e. 2407 or 5000), and primaryChannelNumber is the
// channel number in the range [1, 200].
 required uint32 freq = 8;

// @see #qsh_wifi_channel_width
required uint32 channel_width = 9;

// If channel width is 20MHz, Do not set this field part of message.
// If the channel width is 40, 80, or 160 MHz, then this denotes the channel
// center frequency (in MHz). If the channel is 80+80 MHz, then this denotes
// the center frequency of segment 0, which contains the primary channel.
// This value is derived from the frequency index using the same formula as
// for freq.
optional uint32 cent_freq0 = 10;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 09:36:23 GMT hyungchul.kim@sge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // If the channel width is 80+80MHz, then this denotes the center frequency
    // of segment 1, which does not contain the primary channel. Otherwise, this
    // field is not relevant and set to 0.
    //
    // @see #cent_freq
    optional uint32 cent_freq1 = 11;

    // Flags from qsh_wifi_security_mode_* indicating supported authentication
    // and associated security modes
    // @see qsh_wifi_security_mode
    optional uint32 security_mode = 12;

    // Identifies the radio chain(s) used to discover this AP
    // @see qsh_wifi_radio_chain_flags
    optional uint32 radio_chain = 13;

    // If the QSH_WIFI_RADIO_CHAIN_INDEX_0 bit is set in radio_chain, gives RSSI
    // measured on radio chain 0 in dBm; otherwise invalid and set to 0. This
    // field, along with its relative rssiChain1, can be used to determine RSSI
    // measurements from each radio chain when multiple chains were used to
    // discover this AP.
    // rssi chain 0
    optional int32 rssi_chain0 = 14;
    // rssi chain1
    optional int32 rssi_chain1 = 15;
}

/**
 * Data structure sent with events of type qsh_wifi_event.
 */
message qsh_wifi_evt_scan_result
{
    // Status of event generated in response of scan command
    required qsh_wifi_error status = 1;

    // The total number of results returned by the scan ie this
    // is total number of BSS/AP result found during this scan.
    required uint32 result_total = 2;

    // The qsh wifi implementation may split ap results across multiple
    // events for memory concerns, etc. Hence this number describes the
    // total number of AP's in this event.
    // For e.g. if Wifi Chip Sends 20 AP in single scan to sensor.
    // Now let us say sensor can send max 12 AP in single event then
    // For first event, result_count will be 12 and 8 for second event.
    required uint32 result_count = 3;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // Indicates this is event last in series of event sent by the sensor.
    // This hints client to whether they should wait for more scan events are
    // coming from sensor.
    required bool last_in_series = 4;

    // A value from enum qsh_wifi_scan_type indicating type of scan performed
    optional uint32 scan_type = 5;

    // Timestamp when the Ranging was completed in QTIMER ticks
    optional uint64 ref_time = 6;

    // If not set indicates that all freqs applicable for the scan_type were
    // scanned, else A list containing set of frequencies that were scanned.
    // Frequencies are specified as channel center frequencies in MHz.
    repeated uint32 scanned_freq_list = 7;

    // An AP list containing result_count entries. Will not be present if
    // result_count is 0.
    repeated qsh_wifi_bss_info ap_list = 8;

    // Set to a value from enum qsh_wifi_pref_radio_chain indicating the radio
    // chain preference used for the scan.
    optional qsh_wifi_pref_radio_chain radio_chain_pref = 10;
}

/**
 * Defines action for monitor mode request
 */
enum qsh_wifi_monitor_action
{
    option (nanopb_enumopt).long_names = false;

    // Configured sensor to stop monitor mode scan
    QSH_WIFI_MONTIOR_ACTION_OFF = 0;

    // Configured sensor to start monitor mode scan
    QSH_WIFI_MONTIOR_ACTION_ON  = 1;
}

/**
 * Indicates whether RTT ranging with a specific device succeeded
 */
enum qsh_wifi_rtt_status
{
    option (nanopb_enumopt).long_names = false;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // Ranging completed successfully
    QSH_WIFI_RTT_STATUS_SUCCESS = 0;
    // Ranging failed due to an unspecified error
    QSH_WIFI_RTT_STATUS_FAILURE = 1;
    // target STA does not respond to request
    QSH_WIFI_RTT_STATUS_FAIL_NO_RSP = 2;
    // Ranging request rejected by target
    QSH_WIFI_RTT_STATUS_FAIL_REJECTED = 3;
    // timing measurement times out
    QSH_WIFI_RTT_STATUS_FAIL_TM_TIMEOUT = 4;
    // Target on different channel, cannot range
    QSH_WIFI_RTT_STATUS_FAIL_AP_ON_DIFF_CHANNEL = 5;
    // ranging not supported by target
    QSH_WIFI_RTT_STATUS_FAIL_NO_CAPABILITY = 6;
    // Invalid T1-T4 timestamp
    QSH_WIFI_RTT_STATUS_FAIL_INVALID_TS = 7;
    // responder cannot collaborate at time of request
    QSH_WIFI_RTT_STATUS_FAIL_BUSY_TRY_LATER = 8;
    // bad request args
    QSH_WIFI_RTT_STATUS_INVALID_REQ = 9;
    // WiFi not enabled
    QSH_WIFI_RTT_STATUS_NO_WIFI = 10;
}

/**
 * Indicates Altitude type, Ref: RFC 6225 2.4
 */
enum qsh_wifi_lci_altitude_type
{
    option (nanopb_enumopt).long_names = false;

    QSH_WIFI_LCI_ALTITUDE_TYPE_UNKNOWN = 0;
    QSH_WIFI_LCI_ALTITUDE_TYPE_METERS  = 1;
    QSH_WIFI_LCI_ALTITUDE_TYPE_FLOORS  = 2;
}

/**
 * Indicates Result is carry additional info apart from standard
 * ranging results ie like LCI is present in BSS results
 */
enum qsh_rtt_result_has
{
    option (nanopb_enumopt).long_names = false;
    QSH_RTT_RESULT_HAS_LCI = 0x00000001;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungwul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
/**
 * Message to requeset sensor to start/stop monitor scan
 */
message qsh_wifi_cmd_monitor_req
{
    // see @qsh_wifi_monitor_action
    required qsh_wifi_monitor_action action = 1;
}

/**
 * Message to Send Ranging request to sensor from clients
 *
 * To keep the interface simple below things are assumed
 * In future, on need basis we can add additional params.
 *
 * Ranging will be always two sided
 * ASAP field will always  be true for measurment request
 * BSS measurments will be single shot in nature
 * BSS PEER Type will be set to as ACCESS POINT (AP)
 * Max Burst duration will be 15 ie no preference
 * Number of retries per bssid measurement is 0
 *
 */
message qsh_wifi_cmd_ranging_req
{
    // MAX number of ranging node shoule be 20.
    repeated qsh_wifi_ranging_req_ap bss_list = 1;
}

/**
 * Identifies a device to perform RTT ranging against.
 * These values are normally
 * populated based on the contents of a scan result.
 */
message qsh_wifi_ranging_req_ap
{
    // MAC address of the device for which ranging was requested
    required bytes mac_addr = 1[(nanopb).max_size = 6];

    // Indicates the center frequency of the primary 20MHz channel
    required uint32 freq = 2;

    // @see #qsh_wifi_channel_width
    required qsh_wifi_channel_width channel_width = 3;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // If channel width is 20MHz, this field is ignored.
    // If the channel width is 40, 80, or 160 MHz, then this denotes the channel
    // center frequency (in MHz). If the channel is 80+80 MHz, then this denotes
    // the center frequency of segment 0, which contains the primary channel.
    optional uint32 cent_freq0 = 4;

    // If the channel width is 80+80MHz, then this denotes the center frequency
    // of segment 1, which does not contain the primary channel.
    optional uint32 cent_freq1 = 5;

    // Get the LCI data along with ranging data if responder have it
    required bool request_lci = 6;

    // num of frames per burst.
    // Minimum value = 1, Maximum value = 31
    // For 2-sided this equals the number of FTM frames
    // to be attempted in a single burst. This also
    // equals the number of FTM frames that the
    // initiator will request that the responder send
    // in a single frame.
    required uint32 num_frames_per_burst = 7 [default = 8];
}

/**
 * message for sending Ranging Results/Events to client.
 */
message qsh_wifi_evt_ranging_result
{
    // Status of event generated in response of scan command
    required qsh_wifi_error status = 1;

    // The total number of results returned by the scan ie this
    // is total number of BSS/AP result found during this scan.
    required uint32 result_total = 2;

    // The qsh wifi implementation may split ap results across multiple
    // events for memory concerns, etc. Hence this number describes the
    // total number of AP's in this event.
    // For e.g. if Wifi Chip Sends 20 AP in single scan to sensor.
    // Now let us say sensor can send max 12 AP in single event then
    // For first event, result_count will be 12 and 8 for second event.
    required uint32 result_count = 3;

    // Indicates this is event last in series of event sent by the sensor.
    // This hints client to whether they should wait for more scan events are
    // coming from sensor.
    required bool last_in_series = 4;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungsul.woo@line.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // An AP list containing result_count entries. Will not be present if
    // result_count is 0.
    repeated qsh_wifi_ranging_result_ap bss_list = 5;
}

/**
 * Provides the result of RTT ranging with a single device.
 */
message qsh_wifi_ranging_result_ap
{
    // MAC address of the device for which ranging was requested
    required bytes mac_addr = 1[(nanopb).max_size = 6];

    // Gives the result of ranging to this device. If not set to
    // qsh_wifi_rtt_status_success, the ranging attempt to this device
    // failed, and other fields in this structure may be invalid.
    // @see #qsh_wifi_rtt_status
    required qsh_wifi_rtt_status rtt_status = 2;

    // Set of RTT Result flags, see qsh_rtt_result_has bitwise OR of
    // qsh_rtt_result_has bits.
    required uint32 result_flag = 3;

    // Timestamp when the Ranging was completed in QTIMER ticks
    optional uint64 timestamp = 4;

    // The mean RSSI measured during the RTT burst, in dBm. Typically negative.
    // If rtt_status is not qsh_wifi_rtt_status_success then it won't present.
    optional int32 rssi = 5;

    // Estimated distance to the device with the given BSSID, in millimeters.
    // Generally the mean of multiple measurements performed in a single burst.
    // If rtt_status is not qsh_wifi_rtt_status_success then it won't present.
    optional int32 distance = 6;

    // Standard deviation of estimated distance across multiple measurements
    // performed in a single RTT burst, in millimeters. If status is not
    // If rtt_status is not qsh_wifi_rtt_status_success then it won't present.
    optional int32 distance_stddev = 7;

    // Latitude in degrees as 2's complement fixed-point with 25 fractional
    // bits, i.e. degrees * 2^25. Ref: RFC 6225 2.3
    // it will be only present  if flag have bit set QSH_RTT_RESULT_HAS_LCI.
    optional int64 latitude = 8;

    // Longitude, same format as latitude
    // it will be only present  if flag have bit set QSH_RTT_RESULT_HAS_LCI.
    optional int64 longitude = 9;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:35:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    // Altitude represented as a 2's complement fixed-point value with 8
    // fractional bits. Interpretation depends on qsh_wifi_lci_altitude_type.
    // If UNKNOWN, this field must be ignored. If *METERS, distance relative
    // to the zero point in the vertical datum. If *FLOORS, a floor value
    // relative to the ground floor, potentially fractional, e.g. to
    // indicate mezzanine levels. Ref: RFC 6225 2.4
    // it will be only present  if flag have bit set QSH_RTT_RESULT_HAS_LCI.
    optional int32 altitude = 10;

    // Maximum extent of latitude uncertainty in degrees, decoded via this
    // formula: 2 ^ (8 - x) where "x" is the encoded value passed in this
    // field. Unknown if set to 0.
    // Ref: RFC 6225 2.3.2
    // it will be only present  if flag have bit set QSH_RTT_RESULT_HAS_LCI.
    optional uint32 latitude_uncertainty = 11;

    // @see #latitudeUncertainty
    // it will be only present  if flag have bit set QSH_RTT_RESULT_HAS_LCI.
    optional uint32 longitude_uncertainty = 12;

    // Defines how to interpret altitude, set to a value from enum
    // qsh_wifi_lci_altitude_type
    // it will be only present  if flag have bit set QSH_RTT_RESULT_HAS_LCI.
    optional qsh_wifi_lci_altitude_type altitude_type = 13;

    // Uncertainty in altitude, decoded via this formula: 2 ^ (21 - x)
    // where "x" is the encoded value passed in this field. Unknown if set
    // to QSH_WIFI_LCI_UNCERTAINTY_UNKNOWN. Only applies when altitude_type
    // is QSH_WIFI_LCI_ALTITUDE_TYPE_METERS. Ref: RFC 6225 2.4.5
    // it will be only present if flag have bit set QSH_RTT_RESULT_HAS_LCI.
    optional uint32 altitude_uncertainty = 14;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 09:35:22 GMT hyungchul.kim@ae.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

10 QSH WWAN (modem) API

QSH WWAN (modem) API is available at adsp_proc/qsh/qsh_wwan/pb/qsh_wwan.proto in the ADSP build.

```
/* @file qsh_wwan.proto
 *
 * API for the WWAN virtual sensor. The WWAN sensor provides an API to query
 * information about the serving and neighboring cells. The information includes
 * parameters that constitute the identity of the cells, and parameters that
 * characterize the signal/channel from the perspective of the UE.
 *
 * Copyright (c) 2020 Qualcomm Technologies, Inc.
 * All Rights Reserved.
 * Confidential and Proprietary - Qualcomm Technologies, Inc. */

syntax = "proto2";
import "nanopb.proto";
import "sns_std_sensor.proto";

// WWAN Attribute Requirements:
// SNS_STD_SENSOR_ATTRID_TYPE: "wwan"
// SNS_STD_SENSOR_ATTRID_AVAILABLE: true
// SNS_STD_SENSOR_ATTRID_VERSION: 0x00010000
// SNS_STD_SENSOR_ATTRID_API: "qsh_wwan.proto"
// SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_ON_CHANGE

/* Events from the WWAN sensor will have the
sns_client_event_msg.sns_client_event.msg_id
 * field set to a value in this enum. */
enum qsh_wwan_msgid {
    option (nanopb_enumopt).long_names = false;

    /* Sensors framework requires IDs of non-recurrent events to be in range 768 -
1023. */

    /* Information about the serving and neighboring cells. Field
     * sns_client_event_msg.sns_client_event.payload in the event
     * is of type qsh_wwan_event_serving_neighboring_cell_info. */
    QSH_WWAN_MSGID_QSH_WWAN_EVENT_SERVING_NEIGHBORING_CELL_INFO = 800;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 09:36:23 GMT hyung.hwi.kim@lge.com

```
    /* Information about the high speed rail network identification. */
    QSH_WWAN_MSGID_QSH_WWAN_EVENT_HIGHSPEED_RAIL_NETWORK_INFO = 801;

    /* Provides state updates of voice calls in the system.
     * Field sns_client_event_msg.sns_client_event.payload in the event
     * is of type qsh_wwan_event_voice_calls. */
    QSH_WWAN_MSGID_QSH_WWAN_EVENT_VOICE_CALL_INFO = 802;
}

/* If the optional WWAN sensor configuration is required, it must be provided
 * in the payload of SNS_STD_SENSOR_MSGID_SNS_STD_ON_CHANGE_CONFIG. In other
 * words, if the payload of SNS_STD_SENSOR_MSGID_SNS_STD_ON_CHANGE_CONFIG is
 * not empty, it must be of type qsh_wwan_sensor_config. */
message qsh_wwan_sensor_config {
    /* Report information about neighboring and serving cells.
     * Currently, cell information about neighboring/serving
     * cells is always reported, and this flag has no effect.
     * In the future, this flag will control the reporting of
     * serving/neighboring cell information. */
    optional bool report_cell_info = 1;

    /* Report information about voice calls in the system. If
     * true, voice call information will be reported in event
     * QSH_WWAN_MSGID_QSH_WWAN_EVENT_VOICE_CALL_INFO. */
    optional bool report_voice_call_info = 2;
}

message qsh_wwan_cell_identity_gsm {
    /* 3-digit Mobile Country Code, 0..999, INT32_MAX if unknown */
    required int32 mcc = 1;

    /* 2 or 3-digit Mobile Network Code, 0..999, INT32_MAX if unknown */
    required int32 mnc = 2;

    /* 16-bit Location Area Code, 0..65535, INT32_MAX if unknown */
    required int32 lac = 3;

    /* 16-bit GSM Cell Identity described in TS 27.007, 0..65535,
     * INT32_MAX if unknown */
    required int32 cid = 4;

    /* 16-bit GSM Absolute RF channel number, INT32_MAX if unknown */
    required int32 arfcn = 5;

    /* 6-bit Base Station Identity Code, UINT8_MAX if unknown */
    required uint32 bsic = 6;
}
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@sne.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_wwan_cell_identity_wcdma {
    /* 3-digit Mobile Country Code, 0..999, INT32_MAX if unknown */
    required int32 mcc = 1;

    /* 2 or 3-digit Mobile Network Code, 0..999, INT32_MAX if unknown */
    required int32 mnc = 2;

    /* 16-bit Location Area Code, 0..65535, INT32_MAX if unknown */
    required int32 lac = 3;

    /* 28-bit UMTS Cell Identity described in TS 25.331, 0..268435455,
     * INT32_MAX if unknown */
    required int32 cid = 4;

    /* 9-bit UMTS Primary Scrambling Code described in TS 25.331, 0..511,
     * INT32_MAX if unknown */
    required int32 psc = 5;

    /* 16-bit UMTS Absolute RF Channel Number, INT32_MAX if unknown */
    required int32 uarfcn = 6;
}

message qsh_wwan_cell_identity_cdma {
    /* Network Id 0..65535, INT32_MAX if unknown */
    required int32 network_id = 1;

    /* CDMA System Id 0..32767, INT32_MAX if unknown */
    required int32 system_id = 2;

    /* Base Station Id 0..65535, INT32_MAX if unknown */
    required int32 base_station_id = 3;

    /* Longitude is a decimal number as specified in 3GPP2 C.S0005-A v6.0.
     * It is represented in units of 0.25 seconds and ranges from -2592000
     * to 2592000, both values inclusive (corresponding to a range of -180
     * to +180 degrees). INT32_MAX if unknown */
    required int32 longitude = 4;

    /* Latitude is a decimal number as specified in 3GPP2 C.S0005-A v6.0.
     * It is represented in units of 0.25 seconds and ranges from -1296000
     * to 1296000, both values inclusive (corresponding to a range of -90
     * to +90 degrees). INT32_MAX if unknown */
    required int32 latitude = 5;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:22 GMT hyungchul.yoo@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_wwan_cell_identity_lte {
    /* 3-digit Mobile Country Code, 0..999, INT32_MAX if unknown */
    required int32 mcc = 1;

    /* 2 or 3-digit Mobile Network Code, 0..999, INT32_MAX if unknown */
    required int32 mnc = 2;

    /* 28-bit Cell Identity described in TS ???, INT32_MAX if unknown */
    required int32 ci = 3;

    /* physical cell id 0..503, INT32_MAX if unknown */
    required int32 pci = 4;

    /* 16-bit tracking area code, INT32_MAX if unknown */
    required int32 tac = 5;

    /* 18-bit LTE Absolute RF Channel Number, INT32_MAX if unknown */
    required int32 earfcn = 6;

    /* Cell bandwidth in KHZ, INT32_MAX if unknown */
    required int32 bandwidth = 7;
}

message qsh_wwan_cell_identity_tdscdma {
    /* 3-digit Mobile Country Code, 0..999, INT32_MAX if unknown */
    required int32 mcc = 1;

    /* 2 or 3-digit Mobile Network Code, 0..999, INT32_MAX if unknown */
    required int32 mnc = 2;

    /* 16-bit Location Area Code, 0..65535, INT32_MAX if unknown */
    required int32 lac = 3;

    /* 28-bit UMTS Cell Identity described in TS 25.331, 0..268435455,
     * INT32_MAX if unknown */
    required int32 cid = 4;

    /* 8-bit Cell Parameters ID described in TS 25.331, 0..127, INT32_MAX if
     * unknown */
    required int32 cpid = 5;

    /* 16-bit UMTS Absolute RF Channel Number defined in TS 25.102 5.4.4;
     * INT32_MAX if unknown */
    required int32 uarfcn = 6;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.yoon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_wwan_cell_identity_nr {
    /* 3-digit Mobile Country Code, in range [0, 999]. This value must be valid
     * for registered or camped cells. INT32_MAX means invalid/unreported. */
    required int32 mcc = 1;

    /* 2 or 3-digit Mobile Network Code, in range [0, 999]. This value must be
     * valid for registered or camped cells. INT32_MAX means
     * invalid/unreported. */
    required int32 mnc = 2;

    /* NR Cell Identity in range [0, 68719476735] (36 bits), which
     * unambiguously identifies a cell within a public land mobile network
     * (PLMN). This value must be valid for registered or camped cells.
     * Reference: TS 38.413 section 9.3.1.7. LONG_MAX (2^63-1) means
     * invalid/unreported. */
    required uint64 nci = 3;

    /* Physical cell id in range [0, 1007]. This value must be valid.
     * Reference: TS 38.331 section 6.3.2. */
    required int32 pci = 4;

    /* 24-bit tracking area code in range [0, 16777215]. INT32_MAX means
     * invalid/unreported.
     * Reference: TS 38.413 section 9.3.3.10 and TS 29.571 section 5.4.2. */
    required int32 tac = 5;

    /* NR Absolute Radio Frequency Channel Number, in range [0, 3279165]. This
     * value must be valid.
     * Reference: TS 38.101-1 section 5.4.2.1 and TS 38.101-2 section 5.4.2.1.*/
    required int32 nrarfcn = 6;
}

message qsh_wwan_signal_strength_gsm {
    /* Valid values are (0-31, 99) as defined in TS 27.007 8.5 */
    required int32 signal_strength = 1;

    /* bit error rate (0-7, 99) as defined in TS 27.007 8.5 */
    required int32 bit_error_rate = 2;

    /* Timing Advance in bit periods. 1 bit period = 48.13 us. INT32_MAX
     * denotes invalid value */
    required int32 timing_advance = 3;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@qne.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_wwan_signal_strength_wcdma {
    /* Valid values are (0-31, 99) as defined in TS 27.007 8.5 */
    required int32 signal_strength = 1;

    /* bit error rate (0-7, 99) as defined in TS 27.007 8.5 */
    required int32 bit_error_rate = 2;

    /* CPICH RSCP as defined in TS 25.215 5.1.1
     * Valid values are (0-96, 255) as defined in TS 27.007 8.69
     * INT32_MAX denotes that the value is invalid/unreported. */
    required int32 rscp = 3;

    /* Ec/No value as defined in TS 25.215 5.1.5
     * Valid values are (0-49, 255) as defined in TS 27.007 8.69
     * INT32_MAX denotes that the value is invalid/unreported. */
    required int32 ecno = 4;
}

message qsh_wwan_signal_strength_cdma {
    /* Valid values are positive integers.  This value is the actual RSSI value
     * multiplied by -1.  Example: If the actual RSSI is -75, then this
     * response value will be 75. */
    required int32 dbm = 1;

    /* Valid values are positive integers.  This value is the actual Ec/Io
     * multiplied by -10.  Example: If the actual Ec/Io is -12.5 dB, then this
     * response value will be 125. */
    required int32 ecio = 2;
}

message qsh_wwan_signal_strength_evdo {
    /* Valid values are positive integers.  This value is the actual RSSI value
     * multiplied by -1.  Example: If the actual RSSI is -75, then this
     * response value will be 75. */
    required int32 dbm = 1;

    /* Valid values are positive integers.  This value is the actual Ec/Io
     * multiplied by -10.  Example: If the actual Ec/Io is -12.5 dB, then this
     * response value will be 125. */
    required int32 ecio = 2;

    /* Valid values are 0-8. 8 is the highest signal to noise ratio. */
    required int32 signal_noise_ratio = 3;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungil.kwon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_wwan_signal_strength_lte {
    /* Valid values are (0-31, 99) as defined in TS 27.007 8.5 */
    required int32 signal_strength = 1;

    /* The current Reference Signal Receive Power in dBm multipled by -1.
     * Range: 44 to 140 dBm
     * INT32_MAX: 0x7FFFFFFF denotes invalid value.
     * Reference: 3GPP TS 36.133 9.1.4 */
    required int32 rsrp = 2;

    /* The current Reference Signal Receive Quality in dB multiplied by -1.
     * Range: 20 to 3 dB.
     * INT32_MAX: 0x7FFFFFFF denotes invalid value.
     * Reference: 3GPP TS 36.133 9.1.7 */
    required int32 rsrq = 3;

    /* The current reference signal signal-to-noise ratio in 0.1 dB units.
     * Range: -200 to +300 (-200 = -20.0 dB, +300 = 30dB).
     * INT32_MAX : 0x7FFFFFFF denotes invalid value.
     * Reference: 3GPP TS 36.101 8.1.1 */
    required int32 rssnr = 4;

    /* The current Channel Quality Indicator.
     * Range: 0 to 15.
     * INT32_MAX : 0x7FFFFFFF denotes invalid value.
     * Reference: 3GPP TS 36.101 9.2, 9.3, A.4 */
    required int32 cqi = 5;

    /* timing advance in micro seconds for a one way trip from cell to device.
     * Approximate distance can be calculated using 300m/us * timingAdvance.
     * Range: 0 to 0x7FFFFFFE
     * INT32_MAX : 0x7FFFFFFF denotes invalid value.
     * Reference: 3GPP 36.321 section 6.1.3.5
     * also: http://www.cellular-planningoptimization.com/2010/02/timingadvance-
with-calculation.html */
    required int32 timing_advance = 6;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@qce.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_wwan_signal_strength_tdscdma {
    /* UTRA carrier RSSI as defined in TS 25.225 5.1.4
     * Valid values are (0-31, 99) as defined in TS 27.007 8.5
     * INT32_MAX denotes that the value is invalid/unreported. */
    required int32 signal_strength = 1;

    /* Transport Channel BER as defined in TS 25.225 5.2.5
     * Valid values are (0-7, 99) as defined in TS 27.007 8.5
     * INT32_MAX denotes that the value is invalid/unreported. */
    required int32 bit_error_rate = 2;

    /* P-CCPCH RSCP as defined in TS 25.225 5.1.1
     * Valid values are (0-96, 255) as defined in TS 27.007 8.69
     * INT32_MAX denotes that the value is invalid/unreported. */
    required int32 rscp = 3;
}

message qsh_wwan_signal_strength_nr {
    /* SS (second synchronization) reference signal received power in dBm
     * multiplied by -1.
     * Range [44, 140], INT32_MAX means invalid/unreported.
     * Reference: TS 38.215 section 5.1.1 and TS 38.133 section 10.1.6. */
    required int32 ss_rsrp = 1;

    /* SS reference signal received quality in 0.5 dB units.
     * Range [-86, 41] with -86 = -43.0 dB and 41 = 20.5 dB.
     * INT32_MAX means invalid/unreported.
     * Reference: TS 38.215 section 5.1.3 and TS 38.133 section 10.1.11.1. */
    required int32 ss_rsrq = 2;

    /* SS signal-to-noise and interference ratio in 0.5 dB units.
     * Range [-46, 81] with -46 = -23.0 dB and 81 = 40.5 dB.
     * INT32_MAX means invalid/unreported.
     * Reference: TS 38.215 section 5.1.5 and TS 38.133 section 10.1.16.1. */
    required int32 ss_sinr = 3;

    /* CSI reference signal received power in dBm multiplied by -1.
     * Range [44, 140], INT32_MAX means invalid/unreported.
     * Reference: TS 38.215 section 5.1.2 and TS 38.133 section 10.1.6. */
    required int32 csi_rsrp = 4;

    /* CSI reference signal received quality in 0.5 dB units.
     * Range [-86, 41] with -86 = -43.0 dB and 41 = 20.5 dB.
     * INT32_MAX means invalid/unreported.
     * Reference: TS 38.215 section 5.1.4 and TS 38.133 section 10.1.11.1. */
    required int32 csi_rsrq = 5;
```

Qualcomm Confidential - May Contain Trade secrets 2026-02-27 05:31:23 GMT hyungsul.kwon@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
    /* CSI signal-to-noise and interference ratio in 0.5 dB units.
     * Range [-46, 81] with -46 = -23.0 dB and 81 = 40.5 dB.
     * INT32_MAX means invalid/unreported.
     * Reference: TS 38.215 section 5.1.6 and TS 38.133 section 10.1.16.1. */
    required int32 csi_sinr = 6;
}

message qsh_wwan_cell_info_gsm {
    required qsh_wwan_cell_identity_gsm cell_identity_gsm = 1;
    required qsh_wwan_signal_strength_gsm signal_strength_gsm = 2;
}

message qsh_wwan_cell_info_wcdma {
    required qsh_wwan_cell_identity_wcdma cell_identity_wcdma = 1;
    required qsh_wwan_signal_strength_wcdma signal_strength_wcdma = 2;
}

message qsh_wwan_cell_info_cdma {
    required qsh_wwan_cell_identity_cdma cell_identity_cdma = 1;
    required qsh_wwan_signal_strength_cdma signal_strength_cdma = 2;
    required qsh_wwan_signal_strength_evdo signal_strength_evdo = 3;
}

message qsh_wwan_cell_info_lte {
    required qsh_wwan_cell_identity_lte cell_identity_lte = 1;
    required qsh_wwan_signal_strength_lte signal_strength_lte = 2;
}

message qsh_wwan_cell_info_tdscdma {
    required qsh_wwan_cell_identity_tdscdma cell_identity_tdscdma = 1;
    required qsh_wwan_signal_strength_tdscdma signal_strength_tdscdma = 2;
}

message qsh_wwan_cell_info_nr {
    required qsh_wwan_cell_identity_nr cell_identity_nr = 1;
    required qsh_wwan_signal_strength_nr signal_strength_nr = 2;
}

enum qsh_wwan_cell_time_stamp_type {
    option (nanopb_enumopt).long_names = false;

    QSH_WWAN_CELL_TIME_STAMP_TYPE_UNKNOWN = 0;
    QSH_WWAN_CELL_TIME_STAMP_TYPE_ANTENNA = 1;
    QSH_WWAN_CELL_TIME_STAMP_TYPE_MODEM = 2;
    QSH_WWAN_CELL_TIME_STAMP_TYPE_OEM_RIL = 3;
    QSH_WWAN_CELL_TIME_STAMP_TYPE_JAVA_RIL = 4;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.kim@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
enum qsh_wwan_cell_info_type {
    option (nanopb_enumopt).long_names = false;

    QSH_WWAN_CELL_INFO_TYPE_GSM = 1;
    QSH_WWAN_CELL_INFO_TYPE_CDMA = 2;
    QSH_WWAN_CELL_INFO_TYPE_LTE = 3;
    QSH_WWAN_CELL_INFO_TYPE_WCDMA = 4;
    QSH_WWAN_CELL_INFO_TYPE_TD_SCDMA = 5;
    QSH_WWAN_CELL_INFO_TYPE_NR = 6;
}

message qsh_wwan_cell_info {
    /* Timestamp in nanoseconds */
    required uint64 time_stamp = 1;

    /* A value from enum qsh_wwan_cell_time_stamp_type that identifies
     * the source of the value in timeStamp. This is typically set to
     * QSH_WWAN_CELL_TIME_STAMP_TYPE_OEM_RIL, and indicates that an
     * intermediate module received the data from the modem and forwarded
     * it to the requesting client. */
    required qsh_wwan_cell_time_stamp_type time_stamp_type = 2;

    /* true if this cell is registered, false if not registered */
    required bool registered = 3;

    /* A value from enum qsh_wwan_cell_info_type indicating the radio access
     * technology of the cell, and which one of the cellinfo fields below can
     * be used to retrieve additional information. */
    required qsh_wwan_cell_info_type cell_info_type = 4;

    /* The value in cell_info_type indicates which of the following cell info
     * fields is valid, and should be accessed to get the cell information.
     * Alternatively, depending on the language, a Protobuf compiler generated
     * field or method will be available that identifies which of the fields
     * is valid. For example, in C++, the Protobuf compiler generated method
     * "has_cell_info_lte()" could be used to find out if the "cell_info_lte"
     * field is valid or not. */
    oneof cell_info {
        qsh_wwan_cell_info_gsm cell_info_gsm = 5;
        qsh_wwan_cell_info_cdma cell_info_cdma = 6;
        qsh_wwan_cell_info_lte cell_info_lte = 7;
        qsh_wwan_cell_info_wcdma cell_info_wcdma = 8;
        qsh_wwan_cell_info_tdscdma cell_info_tdscdma = 9;
        qsh_wwan_cell_info_nr cell_info_nr = 10;
    }
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 09:30:23 GMT hyunchul.kim@qc.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
message qsh_wwan_event_serving_neighboring_cell_info {
    /* Information about serving and neighboring cells. */
    repeated qsh_wwan_cell_info cells = 1;
}

message qsh_wwan_event_highspeed_rail_network_info {
    /* Information about high speed rail network identification. */
    required bool hst_lte = 1;

    /* Information about high speed rail network identification. */
    required bool hst_nr5g = 2;
}

enum qsh_wwan_voice_call_state {
    option (nanopb_enumopt).long_names = false;

    QSH_WWAN_VOICE_CALL_STATE_ORIGINATING = 1;
    QSH_WWAN_VOICE_CALL_STATE_INCOMING = 2;
    QSH_WWAN_VOICE_CALL_STATE_CONVERSATION = 3;
    QSH_WWAN_VOICE_CALL_STATE_CC_IN_PROGRESS = 4;
    QSH_WWAN_VOICE_CALL_STATE_ALERTING = 5;
    QSH_WWAN_VOICE_CALL_STATE_HOLD = 6;
    QSH_WWAN_VOICE_CALL_STATE_WAITING = 7;
    QSH_WWAN_VOICE_CALL_STATE_DISCONNECTING = 8;
    QSH_WWAN_VOICE_CALL_STATE_END = 9;
    QSH_WWAN_VOICE_CALL_STATE_SETUP = 10;
    QSH_WWAN_VOICE_CALL_STATE_PRE_ALERTING = 11;
}

message qsh_wwan_event_voice_calls {
    repeated qsh_wwan_voice_call calls = 1;
}

message qsh_wwan_voice_call {
    required uint32 call_id = 1;
    required qsh_wwan_voice_call_state call_state = 2;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.kim@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

11 Presence Detect API

Presence Detect API is available at adsp_proc/ssc_api/pb/sns_presence_detect.proto in the ADSP build.

```
// @file sns_presence_detect.proto
//
// Defines message types for the Presence Dectect Sensor.
//
// Copyright (c) 2022 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.

syntax = "proto2";
import "nanopb.proto";
import "sns_presence.proto";
import "sns_std_sensor.proto";

// The Presence detect Sensor detects if the user is nearby ( within 6 feet ) from
the device
// The timestamp of the event indicates the time the device detected the current
state

// Presence detect Sensor Attribute Requirements:
// SNS_STD_SENSOR_ATTRID_TYPE: "presence_detect"
// SNS_STD_SENSOR_ATTRID_STREAM_TYPE: SNS_STD_SENSOR_STREAM_TYPE_ON_CHANGE

// Stream Requests:
// - SNS_STD_SENSOR_MSGID_SNS_STD_ON_CHANGE_CONFIG is used to enable the sensor

// Stream events:
//
// The sns_presence_detect_event message is used to publish change in user
presence
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 09:20:23 GMT hyungchulwon@qq.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Presence API is available at adsp_proc/ssc_api/pb/sns_presence.proto in the ADSP build.

```
// @file sns_presence.proto
//
// Defines API for the Presence Sensor.
//
// Copyright (c) 2022 Qualcomm Technologies, Inc.
// All Rights Reserved.
// Confidential and Proprietary - Qualcomm Technologies, Inc.

syntax = "proto2";
import "nanopb.proto";

// Message IDs for Presence Sensor
enum sns_presence_msgid {
  option (nanopb_enumopt).long_names = false;

  SNS_PRESENCE_MSGID_SNS_PRESENCE_EVENT = 928;
}
enum sns_presence_event_type
{
  option (nanopb_enumopt).long_names = false;
  SNS_PRESENCE_UNKNOWN          = 0;
  SNS_PRESENCE_DETECTED         = 1;
  SNS_PRESENCE_NOT_DETECTED     = 2;
}

// Events types from Presence sensor
message sns_presence_event
{
  required sns_presence_event_type state = 1 [default = SNS_PRESENCE_UNKNOWN];
}
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:36:23 GMT hyungchul.won@lge.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A References

A.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
SM8650 Sensors Overview	80-40939-24
SM8650 Qualcomm Sensing Hub (QSH) 4.0 Overview	80-40939-17

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:36:23 GMT
hyungchul.won@lge.com